FIAP ON

IOS

PRIMEIROS PASSOS COM APLICATIVOS

4

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 – MVC 7
Figura 2 – Ciclo de Vida (app ainda não foi aberto) 11
Figura 3 – Ciclo de Vida (app fechado ou em background) 12
Figura 4 – Ciclo de Vida (app em background) 12
Figura 5 – Ciclo de Vida (saindo do app) 13
Figura 6 – Ciclo de Vida de uma tela (view) 14
Figura 7 – Inserindo cena dentro de Navigation Controller 16
Figura 8 – Inserindo nova cena 17
Figura 9 – Atribuindo uma controller a uma cena 18
Figura 10 – Adicionando um Botão na Tela 18
Figura 11 – Criando um vínculo entre cenas 19
Figura 12 – Alguns componentes visuais do UIKit 21
Figura 13 – Exemplo de uma View inserida dentro da View principal 22
Figura 14 – Manipulando o frame de uma view 23
Figura 15 – Exemplos de UILabel 25
Figura 16 – Exemplos de UIButton 26
Figura 17 – UItextField sendo utilizado 27
Figura 18 – UItextField 28
Figura 19 – Tela com Button, Labels e Text Fields 30
Figura 20 – Assistant editor 31
Figura 21 – Criando IBOutlet 31
Figura 22 – IBOutlets criados 32
Figura 23 – IBAction criada 33
Figura 24 – Criando a segue a partir do DragonBall 37
Figura 25 – Definindo identifier da segue 37
Figura 26 – Tab Bar Controller 39
Figura 27 – Definindo como Initial View Controller 40
Figura 28 – Adicionando nova cena à Tab Bar Controller 41

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Classe ViewController.....15
Código-fonte 2 – Classe NewViewController.....20
Código-fonte 3 – Manipulando outlets e actions.....33
Código-fonte 4 – Fechando tela apresentada modalmente.....35
Código-fonte 5 – Executando segues via código 38

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 TODA APLICAÇÃO TEM UM CICLO 5
1.1 Arquitetura MVC 5
1.1.1 Model 6
1.1.2 Controller 6
1.1.3 View 6
1.1.4 Verificando MVC em um App 8
1.2 Ciclo de vida de um App 8
1.2.1 Not Running 9
1.2.2 Inativo 9
1.2.3 Ativo 9
1.2.4 Background 10
1.2.5 Suspenso 10
1.3 Ciclo de vida de uma VIEW 13
1.4 Entendendo UIKit, UIView e UIViewController 20
1.4.1 UIKit 20
1.4.2 UIView 21
1.4.3 UIViewController 23
2 COMPONENTES UILABEL E UIBUTTON 24
2.1 UILabel 24
2.2 UIButton 25
2.3 Recuperando informações com UITextField 27
2.4 Interagindo com o usuário: IBActions e IBOutlets 29
2.4.1 IBOutlet 30
2.4.2 IBAction 32
2.5 Navegação entre telas (Modal, Navigation Controllers e TabBarControllers) 34
2.5.1 Segue 34
2.5.2 Modal 35
2.5.3 Navigation Controller 35
2.5.4 Tab Bar Controller 38
REFERÊNCIAS 42

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 TODA APLICAÇÃO TEM UM CICLO

Em pouco tempo, você criou seu primeiro aplicativo iOS e aposto que está simpatizando com a linguagem *Swift.* Daqui para frente, novos recursos serão apresentados e você será capaz de criar aplicações mais completas. Neste capítulo, veremos na prática o funcionamento de um App na arquitetura MVC com os principais elementos de texto, botões e múltiplas telas.

1.1 Arquitetura MVC

Em programação existe um termo muito utilizado que se chama **Design Pattern** que, em suma, são soluções padronizadas para problemas comuns, ou seja, certos tipos de desafios que você costumeiramente enfrenta no seu dia a dia também foram enfrentados por outros programadores e soluções foram encontradas para esses problemas. Essas soluções foram catalogadas, descritas e agora podem ser reutilizadas por quem desejar, e a esse tipo de solução deu-se o nome de Design Pattern, ou seja, Padrão de Criação ou Padrão de Desenvolvimento. **Design Patterns** não são regras que devem ser seguidas, mas sim guias que podem ser usados no desenvolvimento e resolução dos problemas. Quando essas regras fazem parte de uma definição mais ampla, mais arquitetural do seu App, nós a chamamos de **Architectural Patterns**.

Quando produzimos um novo App, criamos muitos códigos e telas, utilizamos componentes para diversos fins e até formulamos nossos próprios. Tudo isso deve ser organizado de uma maneira que seja fácil entender o que cada elemento do seu código pode ou deve fazer, qual papel que cada componente do seu App tem no conjunto. Para facilitar essa organização, a Apple adotou uma arquitetura de desenvolvimento chamada **MVC: Model, View, Controller**, isto é, a forma que sua aplicação é estruturada por padrão segue os princípios definidos por esse **Architectural Patterns.**

O **MVC** foi criado com o intuito de facilitar a separação de papéis dos códigos e componentes de um App. Ele define que cada objeto deve ser atribuído a um dos três papéis: **Model, View ou Controller**. Além dos papéis, define também como estes

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

objetos devem se comunicar uns com os outros e o que devem ou não fazer. Vamos conhecer cada uma dessas camadas:

1.1.1 Model

Os objetos da camada *Model* são responsáveis por encapsular os dados que são específicos de uma aplicação como também definem a lógica computacional que manipula e processa essa informação. Por exemplo, um objeto model poderia representar um estudante em um aplicativo de faculdade ou um personagem em um game, entre milhares de outras coisas. Pense no *model* como sendo o **"o quê"** do seu App. O que é um estudante? O que é um inimigo? Também a lógica que trata esse elemento, ou seja, esses objetos representam a lógica, a inteligência do seu App. Os *models* podem se relacionar com outros *models* na sua aplicação, mas não devem possuir uma conexão direta com o que é apresentada ao usuário, ou melhor, apesar do *model* conter o "**o quê**" ele não é responsável pelo "**como**" essa informação é repassada ao usuário.

1.1.2 Controller

O objeto *controller* é o responsável por definir o "**como**", ou seja, como uma informação deverá ser apresentada ao usuário. Atua como uma ponte entre os *models* de uma aplicação e suas *views,* suas telas, o que o usuário está vendo e com o que está interagindo. Eles também realizam tarefas de configuração e gerenciamento, e controlam o ciclo de vida de outros objetos.

1.1.3 View

É nessa camada onde o usuário realmente interage. Já que a *Model* é o **"o quê"** e a *Controller* é o **"como",** podemos dizer que a *View* seria o "**onde**". Onde você apresentará o resultado de tudo que foi feito e onde o usuário interagirá com o seu *App*. A *view* representa o que está sendo apresentado ao usuário, é a representação visual dos seus *models.* É ela que responde aos toques do usuário, ao que o usuário

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

deseja fazer naquele momento no seu *App*, seja consumir, interagir ou modificar um dado ou uma informação.

Como havia dito anteriormente, o **MVC** não só define os papéis, mas também como interagem. Entender isso é a chave de entender realmente como o **MVC** funciona.

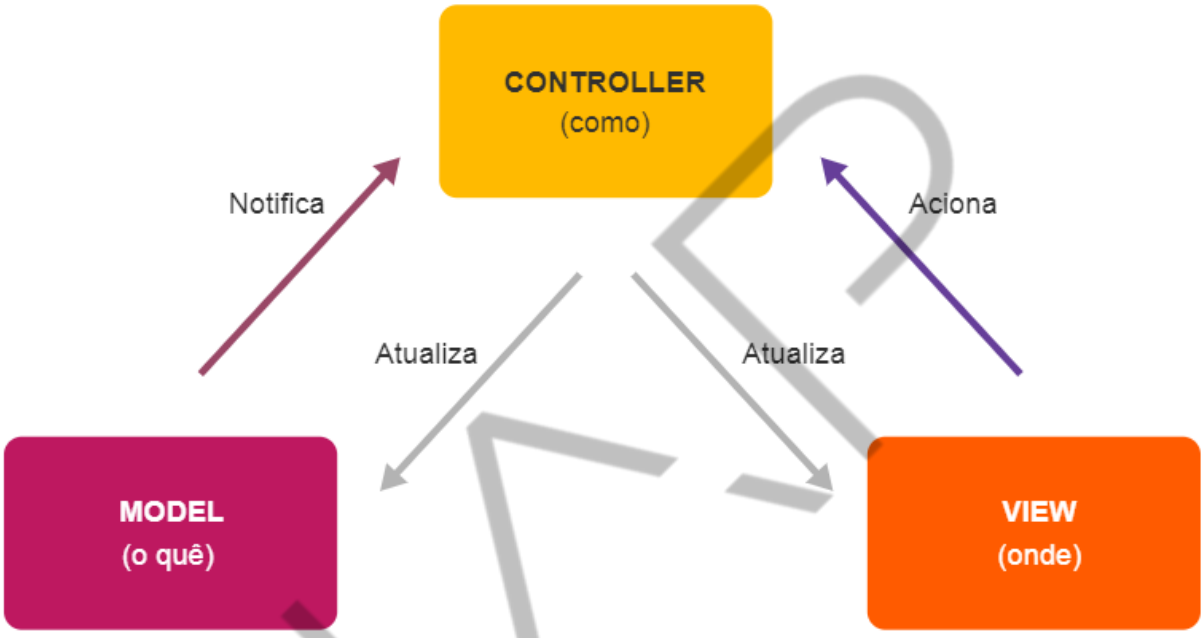

Figura 1 – MVC
Fonte: Elaborado pelo autor (2018)

As interações que o usuário realiza na *view* e que criam/modificam dados são comunicadas diretamente para a *Controller.* A *view* não deve se comunicar com a *Model,* não é seu papel fazer isso. Ela é responsável pela interação com o usuário, ou seja, sabe que o usuário quer fazer algo, mas não é seu papel "fazer" esse algo. Para isso, **aciona** (envia ações) a *Controller,* que é quem realmente define como fazer aquilo que o usuário pediu.

A *Controller* se comunica com os objetos da camada *Model,* manipulando, modificando ou criando tais objetos. Ela **atualiza** os componentes da *model.* Da mesma maneira, quando uma *model* é modificada, quando alguma ação faz com que seus dados sejam atualizados, ela não avisa diretamente a *View* dessa modificação. Ela **notifica** a *Controller,* que por sua vez, **atualiza** a *view* para que reflita ao usuário essas modificações.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vale ressaltar que em um *App,* nós não temos um **MVC**, na verdade, nós temos vários. Cada tela do seu *App* conversará com uma *controller* que, por sua vez, manipulará *Models,* então, logo de cara dá para perceber que podemos e teremos vários **MVCs** em um *App* real. Veremos isso agora do ponto de vista de um *App*. Vamos criar um App novo para que tenhamos uma visualização de como cada uma dessas camadas funciona e quem representa cada uma delas.

1.1.4 Verificando MVC em um App

Vamos abrir novamente o projeto *MyApp*. Nele nós temos alguns arquivos criados automaticamente pelo template escolhido e uma classe que nós mesmos criamos. Naquele mesmo *App,* aprendemos sobre *Storyboard* e como ele pode conter todas as telas de um aplicativo. Neste caso, estamos falando da camada *View*, que é a camada onde o usuário interage com o app, ou seja, o arquivo *Main.storyboard* faz parte da camada *View* do **MVC**.

Neste mesmo App nós criamos uma classe chamada **MyClass**, lembra? Neste caso, o que criamos foi um Model, pois esta classe contém a lógica, a inteligência do nosso App. É responsável por trabalhar uma informação para nós (no caso, nos devolve uma mensagem que poderia ter sido repassada ao usuário).

Por fim, manipulamos o arquivo *ViewController.swift,* que contém a classe *ViewController* que herda de *UIViewController.* Em iOS, classes do tipo *UIViewController* são classes que fazem parte da camada *Controller* do projeto, ou seja, cuidam da interação entre a *Model* e a *View*. No nosso exemplo, nós pegamos o *Model* **(MyClass)** e imprimimos sua mensagem na área de *Debug*, porém, o ideal era mostrar essa mensagem na tela do usuário. Esse é o papel da *Controller,* pegar dados da *Model* e atualizar a *View.*

1.2 Ciclo de vida de um App

Uma das coisas mais básicas, porém, essenciais de se entender quando vamos criar um aplicativo é como o mesmo se comporta dentro do sistema operacional. Em outras palavras, entender como funciona o ciclo de vida de um App. Em que momento

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

inicializa? Quando é encerrado? Quando fica em background? E quando retorna? É muito importante saber como reagir a estes eventos, e é o que iremos aprender agora.

Basicamente, um aplicativo pode se encontrar em cinco estados diferentes. Cada um deles nos é informado através dos métodos específicos que são chamados dentro da classe *AppDelegate.* Esta classe é a responsável por nos fornecer informações sobre o *App* referentes ao seu estado em relação ao sistema operacional (iOS). Vamos conhecer estes estados e que funções são chamadas em cada um deles.

1.2.1 Not Running

Not running (não está sendo executado) é o estado que o aplicativo se encontra quando não foi executado, ou estava rodando, mas foi finalizado pelo sistema. Se seu App nunca foi aberto, ou se foi encerrado (quando o usuário segura o botão *home* e faz o *swipe* para cima com o aplicativo) fica nesse estado.

1.2.2 Inativo

Neste estado, o *App* está rodando em primeiro plano, mas ainda não está recebendo eventos. Um *App* costuma ficar nesse estado apenas por um curtíssimo período de tempo, enquanto faz a transição para um estado diferente (estado ativo). Na classe *AppDelegate,* temos um método chamado ***applicationWillEnterForeground*** que é disparado sempre que o aplicativo se encontra nesse estado, e que o desenvolvedor saiba que o seu App está se movendo do estado de *Background* e voltando para *Foreground* (primeiro plano), porém ainda não está ativo.

1.2.3 Ativo

O *App* está rodando em primeiro plano e está recebendo eventos. Este é o estado normal de um *App*, quando está sendo usado pelo usuário, ou seja, o usuário pode interagir com o aplicativo. O método disparado no *AppDelegate* neste cenário é

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

o **applicationDidBecomeActive**, e, com ele podemos fazer qualquer preparação de última hora no *App*, caso necessário.

1.2.4 Background

Quando o *App* está em background está executando códigos, porém a maioria dos *Apps* ficam nesse estado por um período curto, enquanto se preparam para entrar no estado suspenso. Mesmo assim, é possível que o *App* solicite um tempo extra de execução e fique por mais tempo nesse estado. Também é possível, em alguns cenários, que o *App* seja carregado diretamente nesse estado ao invés do estado de inativo.

O método disparado nesse caso é o **applicationDidEnterBackground**, que nos informa que o *App* agora está em background e pode ser suspenso a qualquer momento.

1.2.5 Suspenso

O *App* está no background, mas não está executando nenhum código. O sistema move os *Apps* para esse estado automaticamente, mas não faz nenhum tipo de notificação antes de fazê-lo. Enquanto estiver nesse estado, o *App* continua na memória, mas não executa códigos.

Quando o sistema percebe que está ocorrendo uma falta de memória, pode retirar o *App* desse estado e levá-lo para o estado "Não está sendo executado", para que mais memória seja liberada para o sistema.

Além dos métodos acima, temos mais alguns que são:

- **application:didFinishLaunchingWithOptions**: Este é o primeiro método que é chamado quando uma aplicação acaba de ser aberta. Geralmente, é aqui que inserimos códigos de configuração do nosso App. Um exemplo seria verificar se o App foi aberto algumas vezes para mostrar ou não uma tela de boas-vindas quando for inicializado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **applicationWillResignActive**: Chamado quando o aplicativo está saindo do estado ativo.
- **applicationWillTerminate:** Chamado imediatamente antes do App ser encerrado pelo usuário (fazendo *swipe* para cima após apertar duas vezes no botão home).

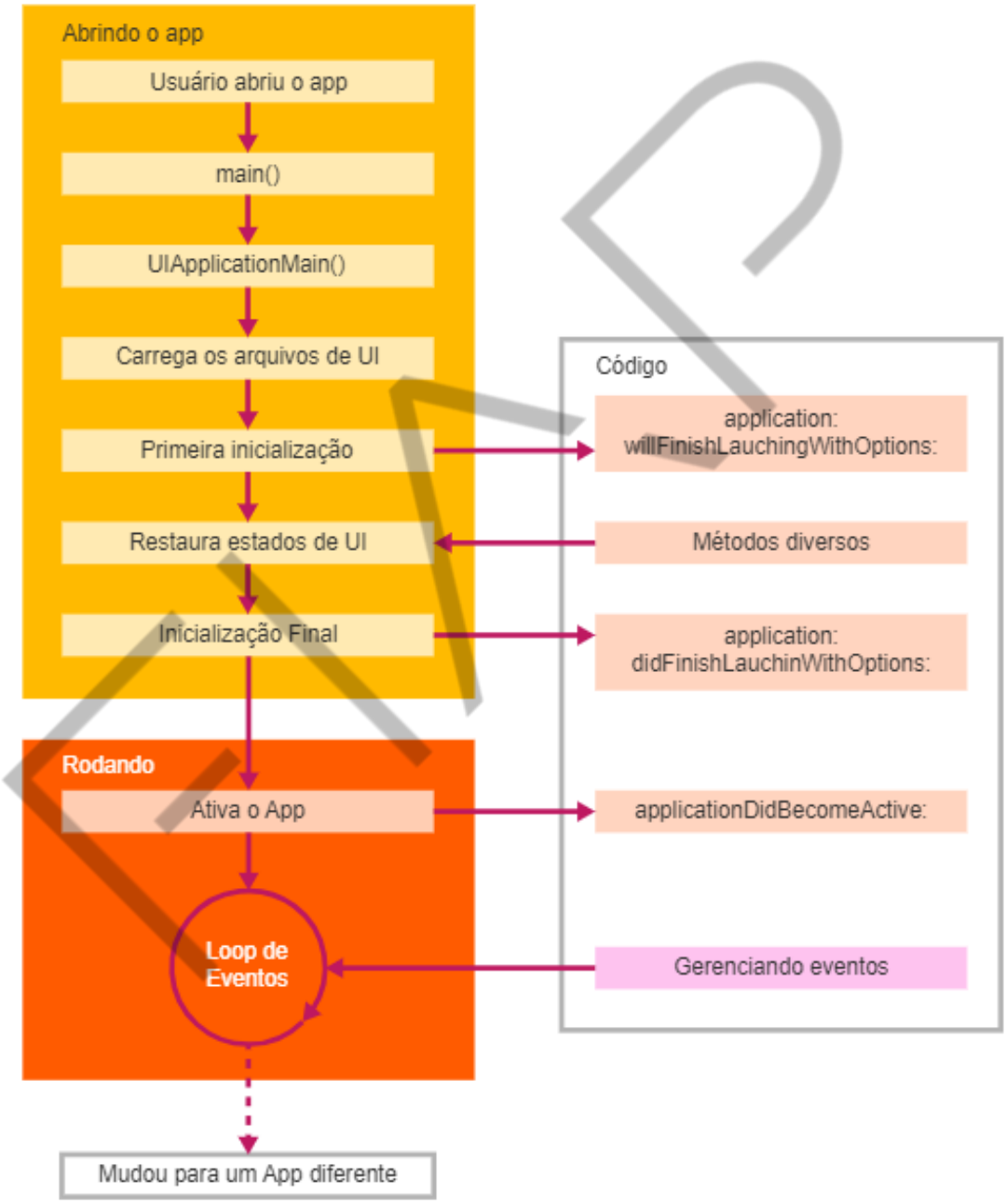

Figura 2 – Ciclo de Vida (app ainda não foi aberto)
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

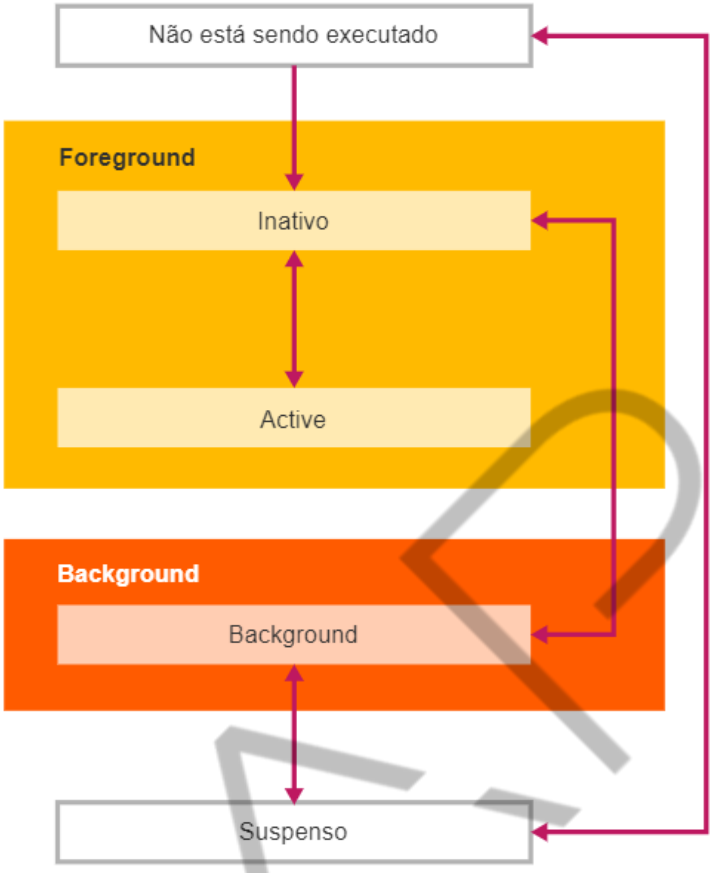

Figura 3 – Ciclo de Vida (app fechado ou em background)
Fonte: Elaborado pelo autor (2018)

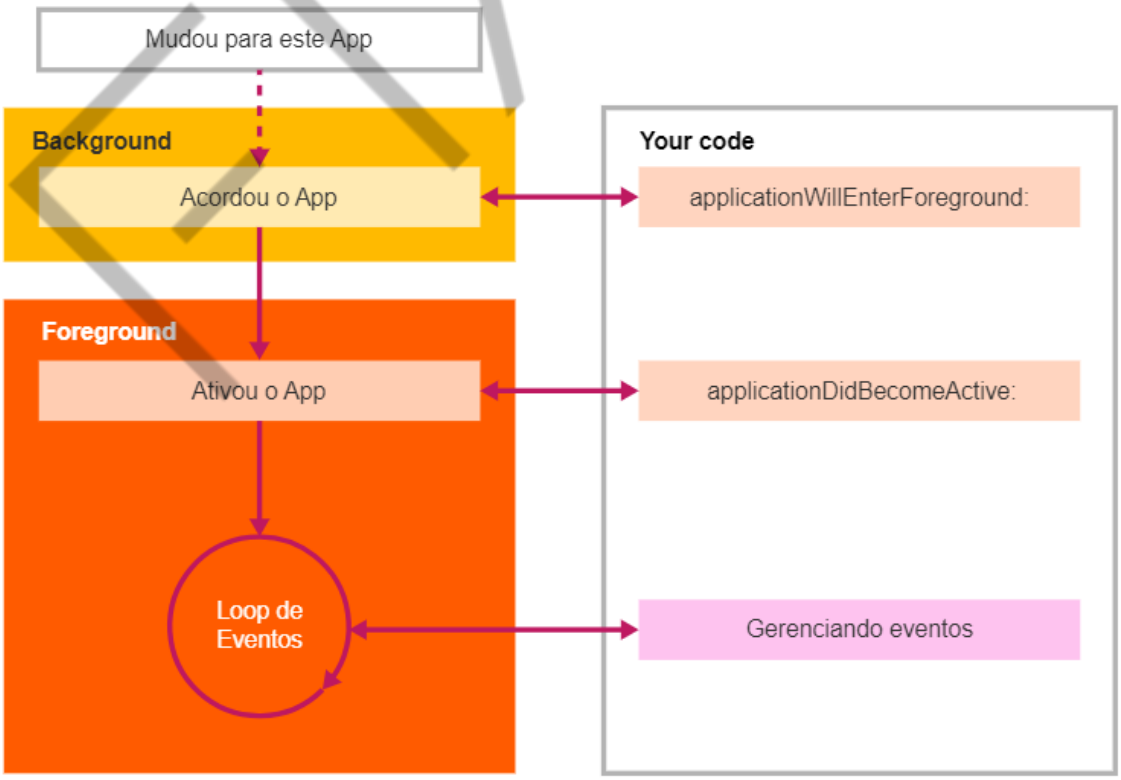

Figura 4 – Ciclo de Vida (app em background)
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

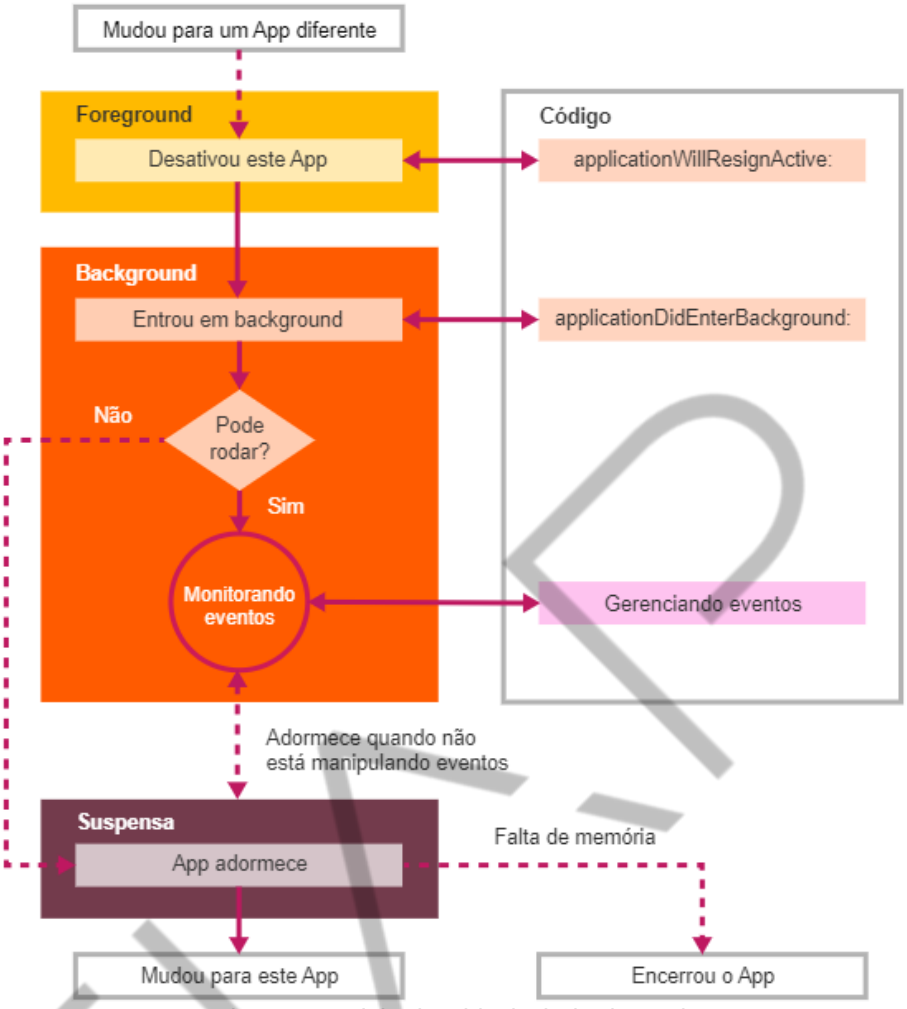

Figura 5 – Ciclo de Vida (saindo do app)
Fonte: Elaborado pelo autor (2018)

1.3 Ciclo de vida de uma VIEW

Assim como o *App*, as telas do aplicativo também possuem um ciclo de vida. É possível saber quando uma tela irá aparecer, quando apareceu, quando irá desaparecer etc. Temos acesso a todos esses eventos através de métodos específicos que são implementados na *controller* da tela em questão. Quando criamos uma *UIViewController,* alguns desses métodos já estão implementados, porém os demais nós precisamos implementar caso desejarmos.

Na figura abaixo temos uma exemplificação de como funciona o ciclo de vida de uma *View* (que está presente em uma cena/tela do App). Usamos o termo *View* pois estes métodos falam sobre o estado da View principal de uma tela, então, podemos assumir que é o próprio ciclo de vida da tela em si.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

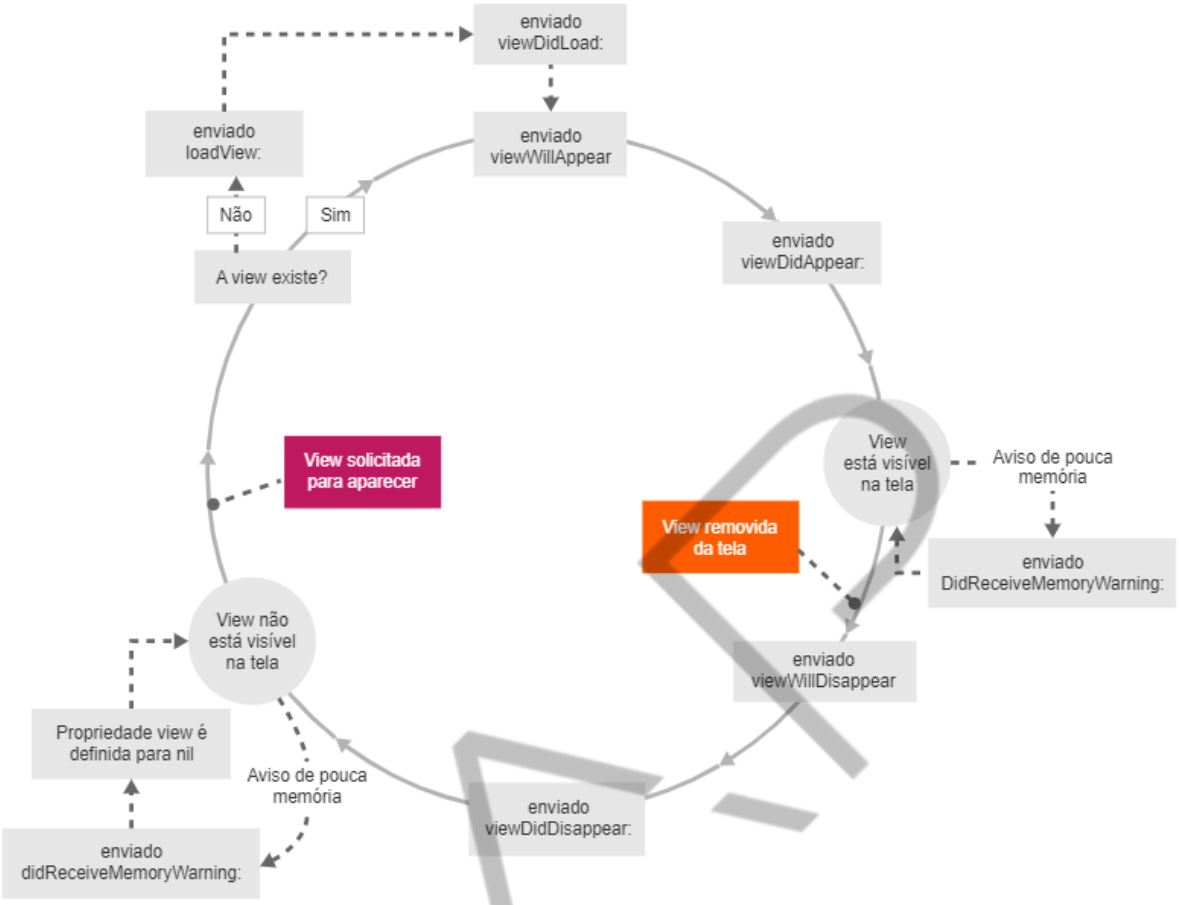

Figura 6 – Ciclo de Vida de uma tela (view)
Fonte: Elaborado pelo autor (2018)

Vimos pela figura anterior que sempre que uma tela é solicitada para aparecer, é verificado se essa tela foi carregada (já se encontra na memória) ou se é a primeira vez que ela aparece. Se a tela ainda não existir, a sua *view* é carregada pelo método **loadView** e na sequência, assim que a *view* é carregada, o método **viewDidLoad** é disparado.

Após ser carregada, a *view* está prestes a aparecer e nesse momento é chamado o método **viewWillAppear** (view irá aparecer). Assim que aparece, o método **viewDidAppear** é chamado (view apareceu). Quando esta tela for dar espaço a uma nova tela, os métodos **viewWillDisappear** (view irá desaparecer) e **viewDidDisappear** (view desapareceu) serão chamados um após o outro, e ao mesmo tempo o ciclo de aparecimento da nova tela ocorre. Um outro método que pode ser chamado é o **didReceiveMemoryWarning**, que é chamado quando o *App* está prestes a ser encerrado pelo sistema por falta de memória. Vale lembrar que nenhum desses métodos é obrigatório de ser implementado. Só implementamos se formos utilizar para algum fim (na grande maioria das vezes o **viewDidLoad** é

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

implementado pois é nesse momento que sabemos que a *view* foi carregada e que podemos manipular os elementos na tela).

Para ver isto, o ciclo de vida das *views* na prática, crie um novo *App* (chame-o de **PrimeirosPassos**) e na classe *ViewController* troque todo o seu código pelo código abaixo:

```
import UIKit

class ViewController: UIViewController {

    override func viewDidLoad() {
        super.viewDidLoad()
        print("Tela 1: viewDidLoad (view foi carregada)")
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        print("Tela 1: viewWillAppear (view vai aparecer)")
    }

    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        print("Tela 1: viewDidAppear (view apareceu)")
    }

    override func viewWillDisappear(_ animated: Bool) {
        super.viewWillDisappear(animated)
        print("Tela 1: viewWillDisappear (view vai
desaparecer)")
    }

    override func viewDidDisappear(_ animated: Bool) {
        super.viewDidDisappear(animated)
        print("Tela 1: viewDidDisappear (view desapareceu)")
    }
}
```

Código-fonte 1 – Classe ViewController
Fonte: Elaborado pelo autor (2018)

Veja que o que fizemos foi implementar todos os métodos do ciclo de vida de uma *view*, e quando cada um deles for chamado, iremos ver na *Debug Area* uma mensagem específica. Note também que todos são *override*, ou seja, estamos sobrescrevendo tais métodos que existem na classe **UIViewController**, e sendo

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

assim, é importante sempre chamar o método da classe mãe (**super**) passando o mesmo parâmetro **animated** que foi recebido.

Agora, abra o *Main.storyboard* e na View selecione o ícone laranja com um quadrado branco dentro (primeiro ícone no topo da tela), que fica em cima da cena. Ele representa a sua *Controller*. Ao clicar nele, vá no menu ***Editor -> Embed In -> Navigation Controller*.** Isso colocará a sua *controller* para ser manipulada por uma outra *controller* especializada em coordenar a navegação entre telas (veremos mais a respeito dela mais tarde).

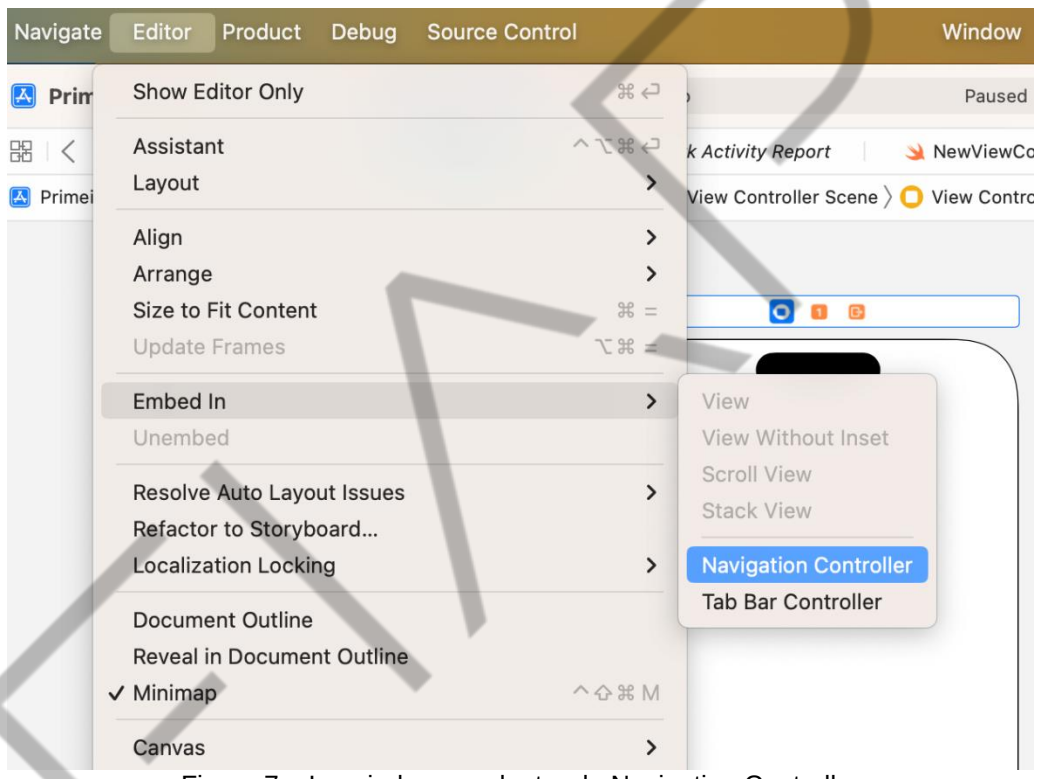

Figura 7 – Inserindo cena dentro de Navigation Controller
Fonte: Elaborado pelo autor (2023)

Agora, adicione uma nova cena no *storyboard.* Para isso, selecione o componente *View Controller* em *Library (é possível acionar o Library pelo atalho* SHIFT+COMMAND+L) e arraste para o *Interface Builder* (deixe ao lado da cena atual).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

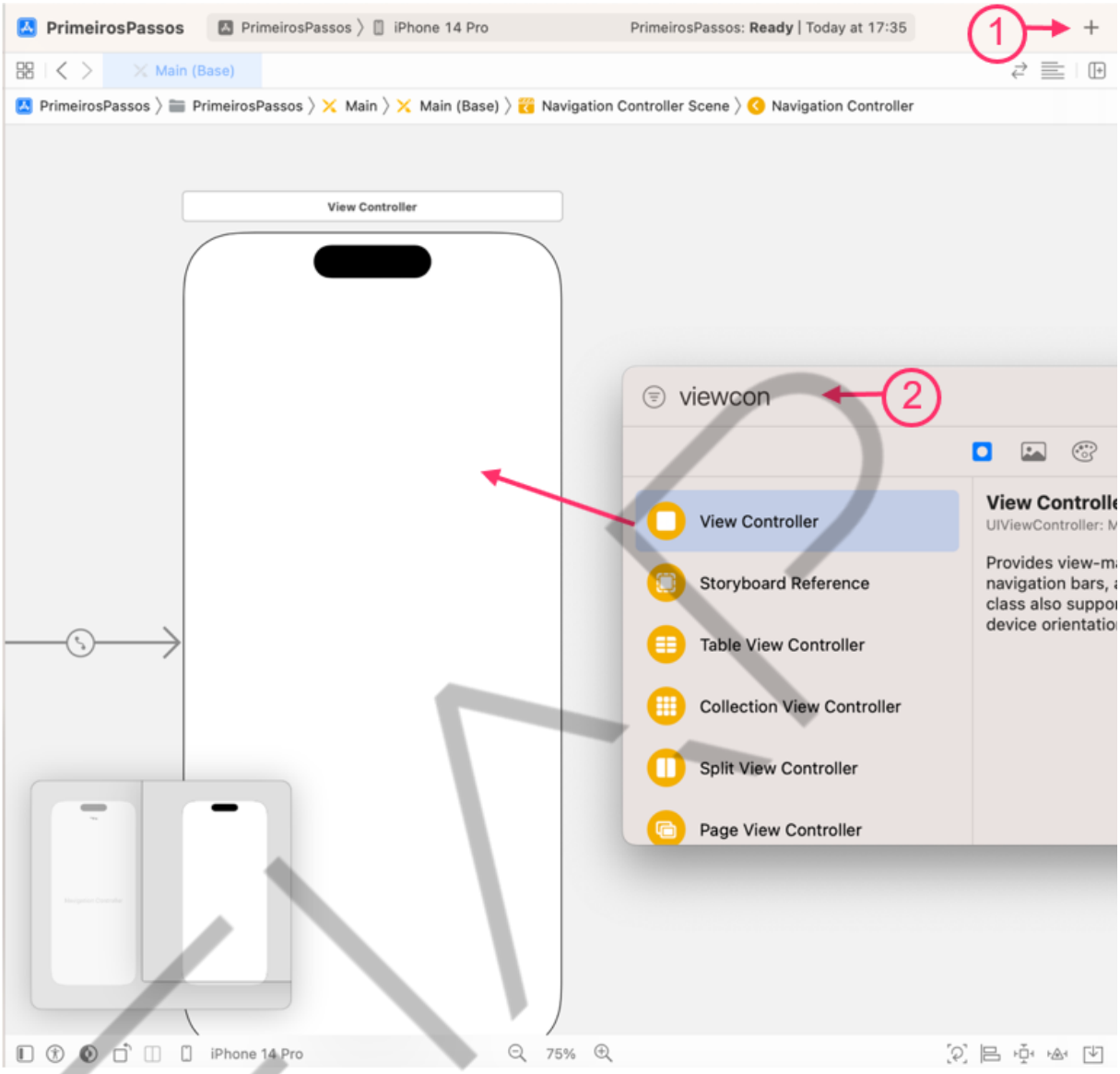

Figura 8 – Inserindo nova cena
Fonte: Elaborado pelo autor (2023)

Vá em **File -> New -> File** e crie uma **Cocoa Touch Class**. Em Class, coloque **NewViewController** e logo abaixo, em Subclass of: escolha **UIViewController**. Com isso, estamos criando uma nova *controller* para nosso aplicativo e essa *controller* será vinculada na cena que acabamos de criar.

Para fazer este vínculo, volte ao *storyboard,* selecione o ícone laranja da nova cena (que iremos apelidar carinhosamente de **DragonBall** daqui para frente), escolha o **Identity Inspector** (quarto botão no topo à direita, um antes do Attributes Inspector) e insira **NewViewController** como sendo a classe associada à esta cena. Esse é o procedimento que fazemos sempre que desejamos vincular uma *controller* a uma cena.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

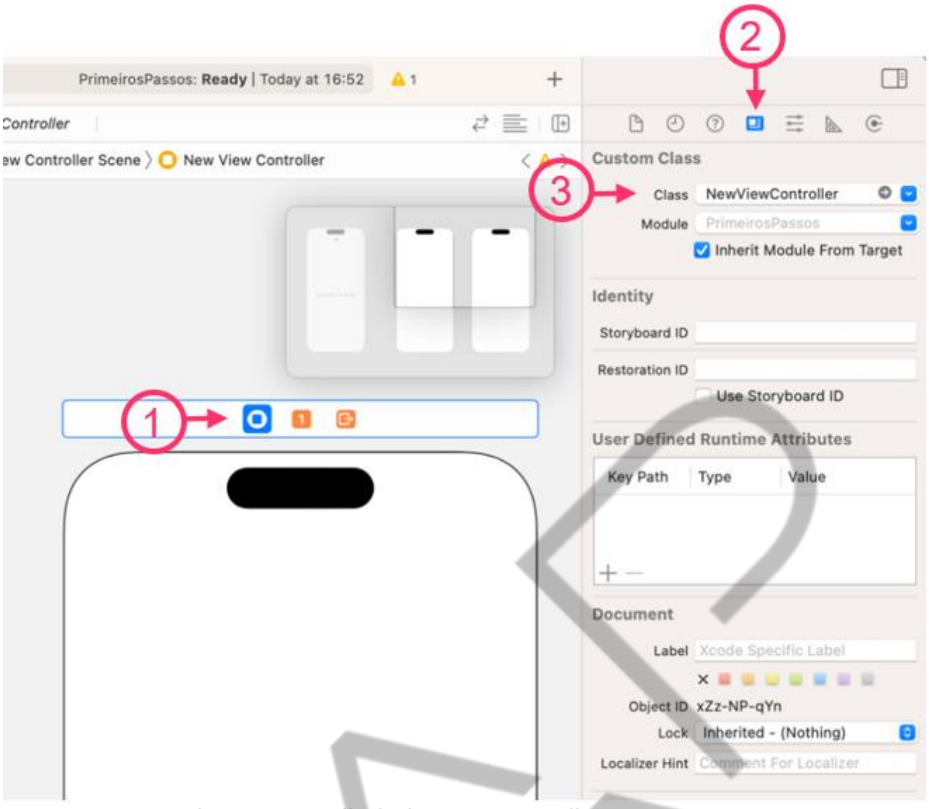

Figura 9 – Atribuindo uma controller a uma cena
Fonte: Elaborado pelo autor (2023)

Agora, adicione um objeto Button (a partir da Library) à sua primeira cena como mostra a imagem abaixo.

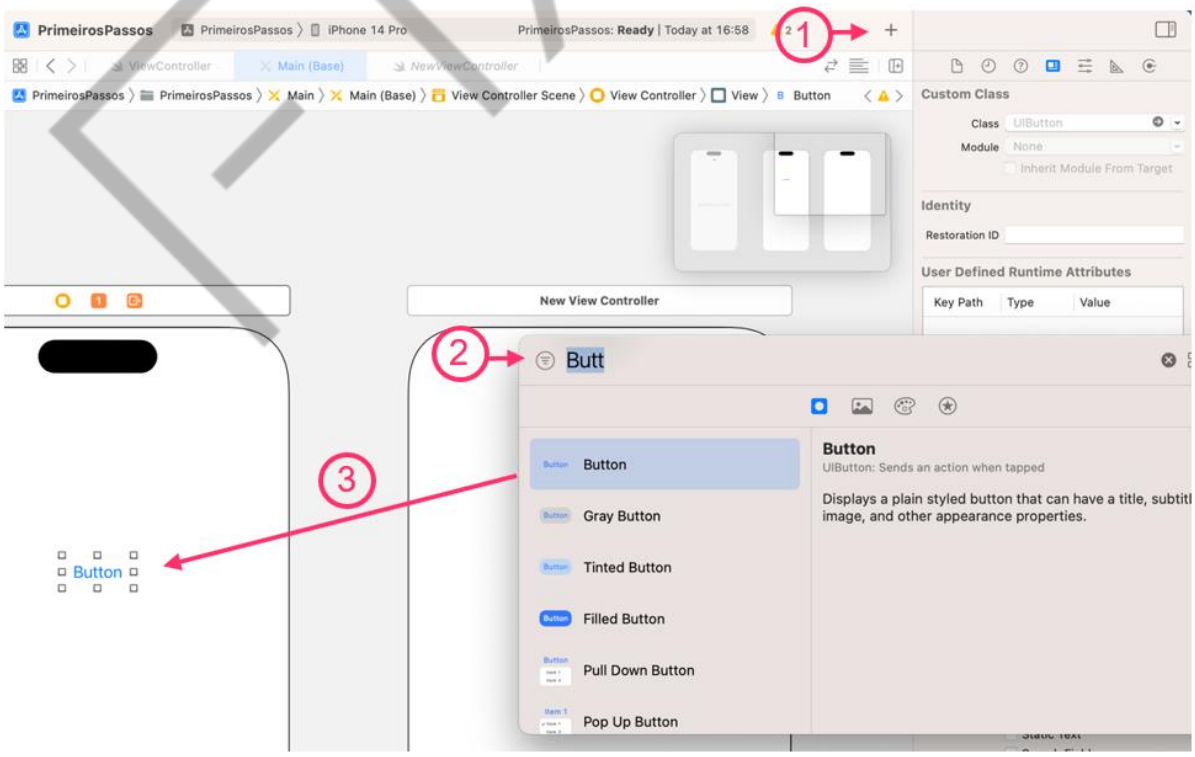

Figura 10 – Adicionando um Botão na Tela
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para finalizar, arraste o Button com o botão direito do mouse (ou *touch pad*) até a segunda cena. No menu que aparecer, selecione a opção **Show**.

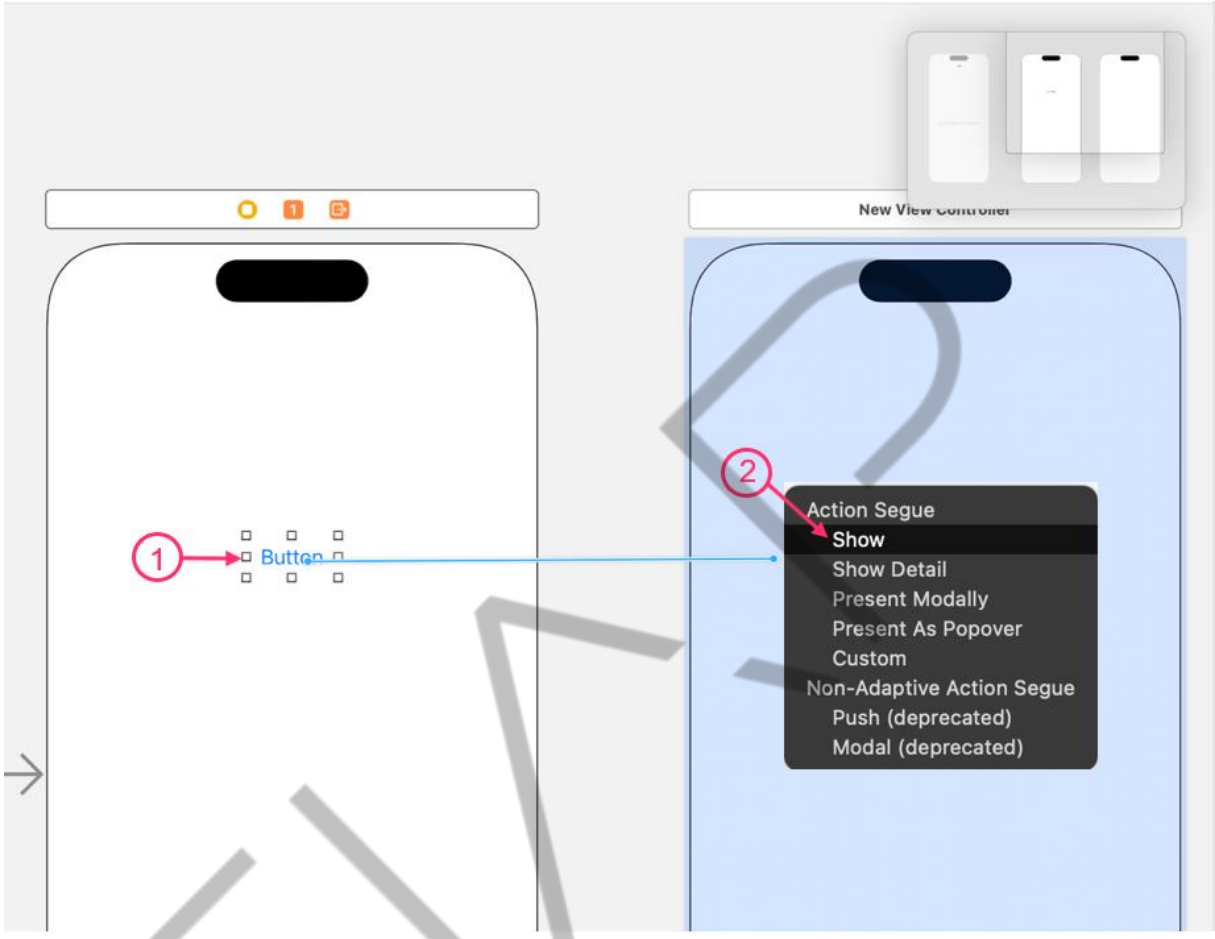

Figura 11 – Criando um vínculo entre cenas
Fonte: Elaborado pelo autor (2023)

Para finalizar, deixe sua classe **NewViewController** com o código abaixo e rode o aplicativo (escolha iPhone 14 Pro como simulador). Você notará que na *Debug Area* aparecerão os eventos referentes ao ciclo de vida das cenas do *App.* Vá de uma cena para outra, volte, repita algumas vezes o processo e analise a saída na *Debug Area.* Você notará cada passo de cada tela dentro do seu App. Outra coisa importante de notar é que a primeira tela só chama o viewDidLoad uma vez. Isso ocorre porque essa tela não sai da memória quando a segunda entra. Agora, a segunda sai quando volta para a primeira, pois essa é a forma que o NavigationController gerencia as telas de um App (veremos mais sobre NavigationController, vínculo entre telas e botões, mais para frente).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import UIKit

class NewViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()
        print("Tela 2: viewDidLoad (view foi carregada)")
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        print("Tela 2: viewWillAppear (view vai aparecer)")
    }

    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        print("Tela 2: viewDidAppear (view apareceu)")
    }

    override func viewWillDisappear(_ animated: Bool) {
        super.viewWillDisappear(animated)
        print("Tela 2: viewWillDisappear (view vai
desaparecer)")
    }

    override func viewDidDisappear(_ animated: Bool) {
        super.viewDidDisappear(animated)
        print("Tela 2: viewDidDisappear (view desapareceu)")
    }
}
```

Código-fonte 2 – Classe NewViewController
Fonte: Elaborado pelo autor (2018)

1.4 Entendendo UIKit, UIView e UIViewController

1.4.1 UIKit

Todos os componentes e classes que usamos no exemplo anterior fazem parte de um Framework que vem disponível para usarmos em nossas aplicações, que é o framework **UIKit**. Ele é responsável por fornecer todas as classes que lidam com a interface do usuário (User Interface, que é de onde vem o **UI** de **UIKit**).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Se você abrir a classe *ViewController,* notará que a primeira linha de código é justamente aquela que serve para importar o framework **UIKit**, para que dessa forma nós possamos ter acesso a, por exemplo, classe **UIViewController**. Além disso, todo componente visual usado em iOS também é definido nesse framework.

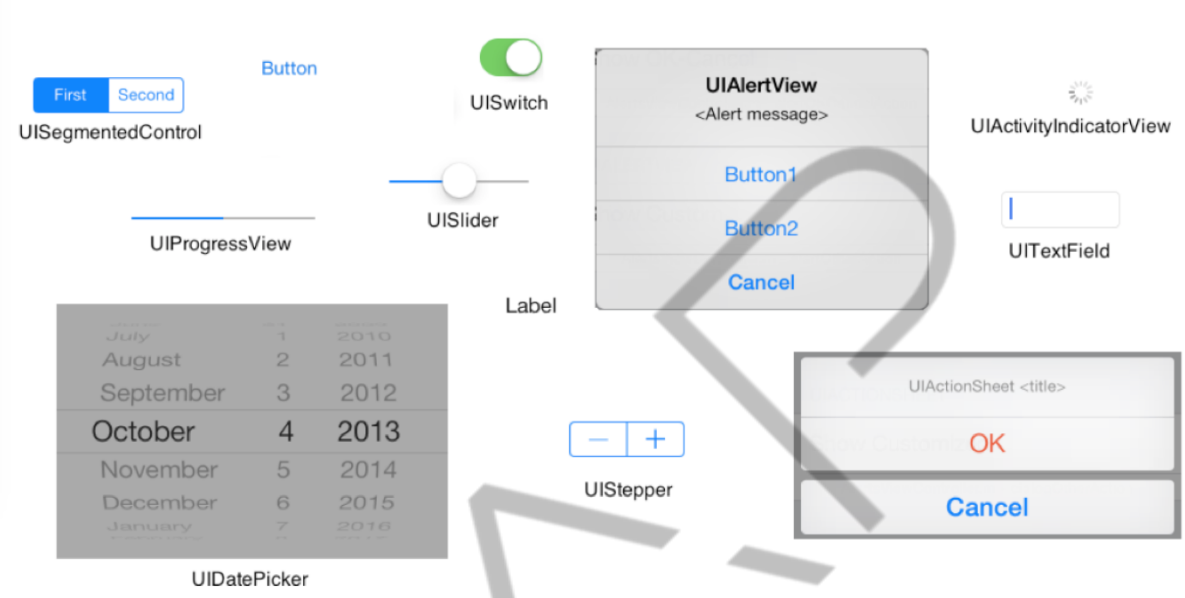

Figura 12 – Alguns componentes visuais do UIKit
Fonte: Elaborado pelo autor (2018)

1.4.2 UIView

UIView é a classe mãe da qual todo e qualquer elemento visual herda. Ela define uma área retangular na tela, além de toda a interface para gerenciamento do conteúdo dessa área. Em iOS, toda tela inicia com uma *view* (UIView), um container que gerencia toda a interface do aplicativo. Você pode adicionar outras *views* dentro da sua cena, para ajudar a organizar os componentes visuais na tela. Dentro de uma *view* podemos adicionar outras *views* (ou outros componentes visuais) e nesse caso passam a ser suas *subviews.* Além disso, é a classe responsável não só pela renderização de quaisquer elementos contidos em sua área, como também a interação com esses elementos. Podemos encontrar o elemento UIView em Library. Dentre suas responsabilidades, destacam-se:

- Renderização e animação.
- Layout e gerenciamento de subviews.
- Gerenciamento de eventos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

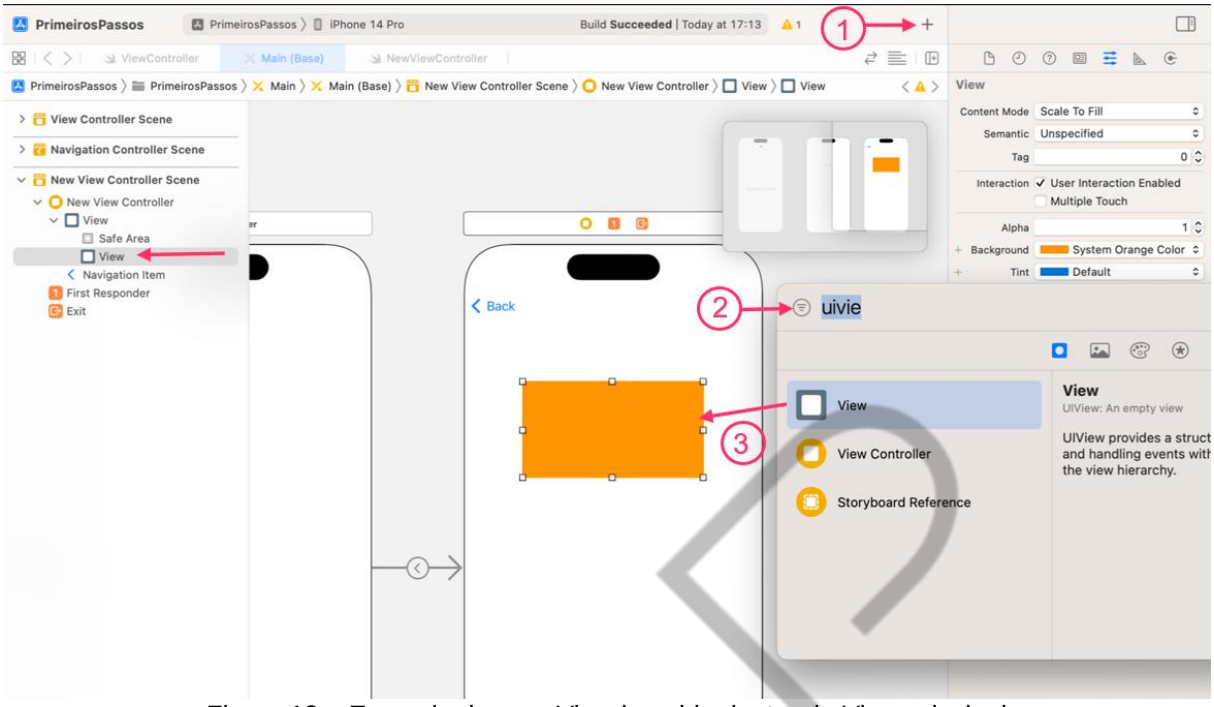

Figura 13 – Exemplo de uma View inserida dentro da View principal
Fonte: Elaborado pelo autor (2023)

O tamanho e a localização de uma *view* na tela (e, consequentemente, qualquer elemento visual) é definido através da propriedade **Frame** que é o retângulo que especifica o tamanho e localização da *view* de acordo com o sistema de coordenadas da sua *superview* (a view na qual ela está inserida). Outra propriedade é a **bounds**, que especifica o tamanho de uma *view* de acordo com seu próprio sistema de coordenadas (bounds é como um frame, só que com a posição **x** e **y** iguais a 0).

O **Frame** possui as propriedades *size*, que definem o seu tamanho, **x** e **y** que definem a sua posição **x** e **y** na tela. Pelo *size* podemos extrair a largura (propriedade width) e altura (propriedade height) da view. Todos esses valores podem ser vistos no *storyboard,* acessando o **Size Inspector**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

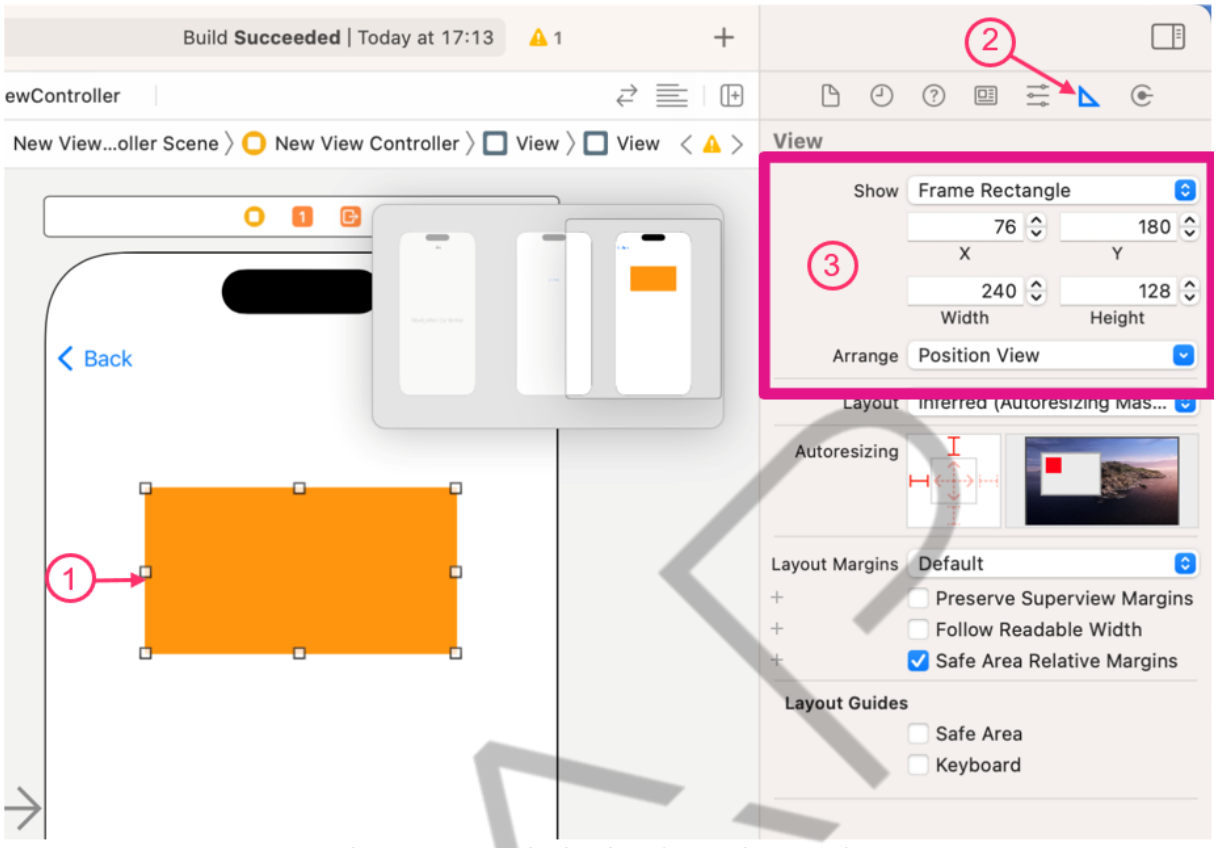

Figura 14 – Manipulando o frame de uma view
Fonte: Elaborado pelo autor (2023)

1.4.3 UIViewController

Esta classe já conhecemos. Toda cena precisa conversar com os *models* do aplicativo, e sabemos que no padrão **MVC** isto ocorre através de uma *controller.* Em iOS, **UIViewController** é a classe mãe de todas as *controllers.*

A classe **UIViewController** define os comportamentos compartilhados que são comuns a todas as *view controllers*. Em desenvolvimento iOS, estamos constantemente criando subclasses de **UIViewController** e adicionando métodos e propriedades necessários para gerenciar a hierarquia de *view* da *view controller.* Dentre suas principais responsabilidades, estão:

- Atualizar o conteúdo das *views*, geralmente em resposta às mudanças ocorridas nos models.
- Responder às interações do usuário nas *views.*
- Gerenciar o layout da interface do usuário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Uma *view controller* está diretamente ligada às *views* que ela gerencia, e responde aos eventos que ocorrem nestas *views*. Cada *App* contém pelo menos uma classe do tipo *view controller,* mas em geral são diversas e na grande maioria das vezes uma para cada cena.

2 COMPONENTES UILABEL E UIBUTTON

2.1 UILabel

Um dos componentes mais utilizados em iOS é o *Label* (classe UILabel). Nele apresentamos informações em formato de texto para o usuário. Já utilizamos Label no nosso aplicativo *HelloUniverse,* inclusive alterando algumas de suas propriedades. Este componente apresenta na tela textos somente para leitura, e podem conter uma ou mais linhas. Dentre as suas propriedades configuráveis estão cor, fonte, alinhamento etc.

Labels podem ser simples (**plain**), que é o estilo padrão e o mais utilizado nos aplicativos, ou com múltiplas configurações (**attributed**). Dentre os seus principais atributos configuráveis via Interface *Builder*, estão:

- **Text**: Define o conteúdo (texto) e o tipo do *Label*. Pode ser *Plain* (texto simples) ou *Attributed* (com múltiplas configurações).
- **Color**: Controla a cor do texto do *label.*
- **Font**: Define a fonte usada no texto do *label.*
- **Alignment**: Controla o alinhamento horizontal do texto. Pode ser left, right, center, justified e/ou natural (definido pelos parâmetros de região do App).
- **Lines**: Define o número máximo de linhas. Para um número indefinido de linhas, utilizamos 0.
- **Line Break**: Define a forma como o texto será tratado quando não couber no seu container. Clip (cortar), *Character/Word Wrap* (quebrar letra ou linha), *Truncate Head/Middle/Tail* (insere três pontos no início, meio ou fim).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como exercício, adicione vários *Labels* na tela e experimente modificar suas diversas propriedades. Tente replicar os diferentes estilos de *labels* do exemplo.

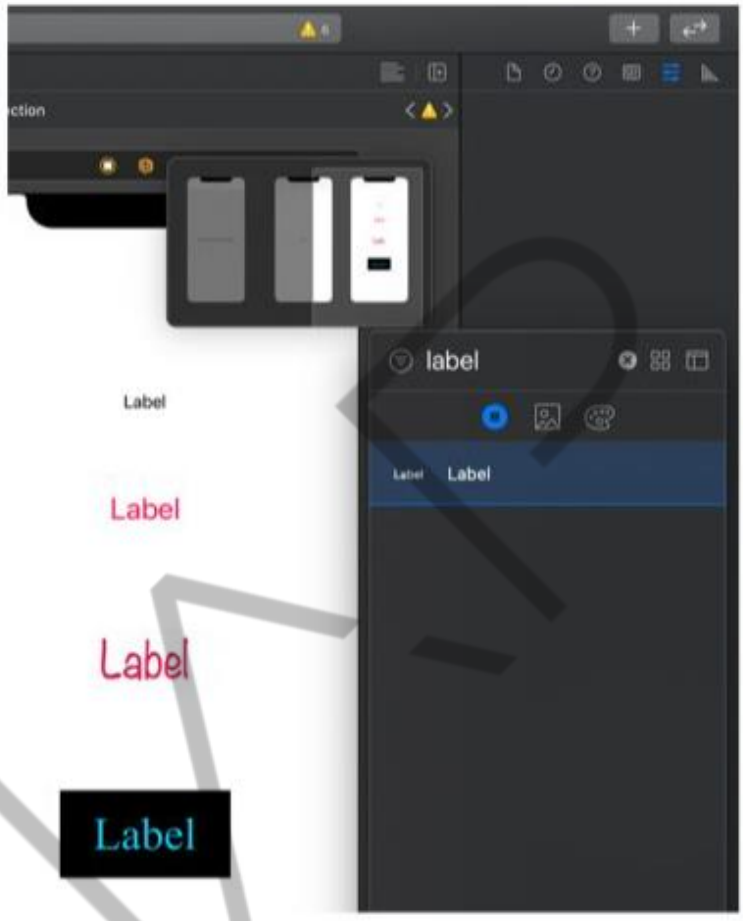

Figura 15 – Exemplos de UILabel
Fonte: Elaborado pelo autor (2021)

2.2 UIButton

Esta classe representa o componente Button (botão), o principal componente que utilizamos quando desejamos executar códigos em resposta à interação do usuário. Ele pode ser configurado para usar textos, imagens *(foreground e background*) ou ambos. Sua aparência pode ser definida através de cinco estados: *default, highlighted, focused, selected e disabled*.

Dentre as principais propriedades configuráveis no *Interface Builder,* estão:

- **Type**: Define o tipo do botão, o que também define os comportamentos padrões do botão para outros atributos. Não pode ser alterado em tempo de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

execução. Dentre os valores, estão *System* (botões padrões do sistema) e *Custom* (botões com características e comportamentos customizados).

- **State Config**: Seleciona o estado do botão, dando acesso às suas configurações.
- **Title**: Título (texto) do botão.
- **Image**: Imagem do botão.
- **Background**: Imagem de fundo do botão. É apresentada atrás do título e imagem, e ocupa toda a área (frame) do botão.

Experimente adicionar vários botões e modifique suas propriedades.

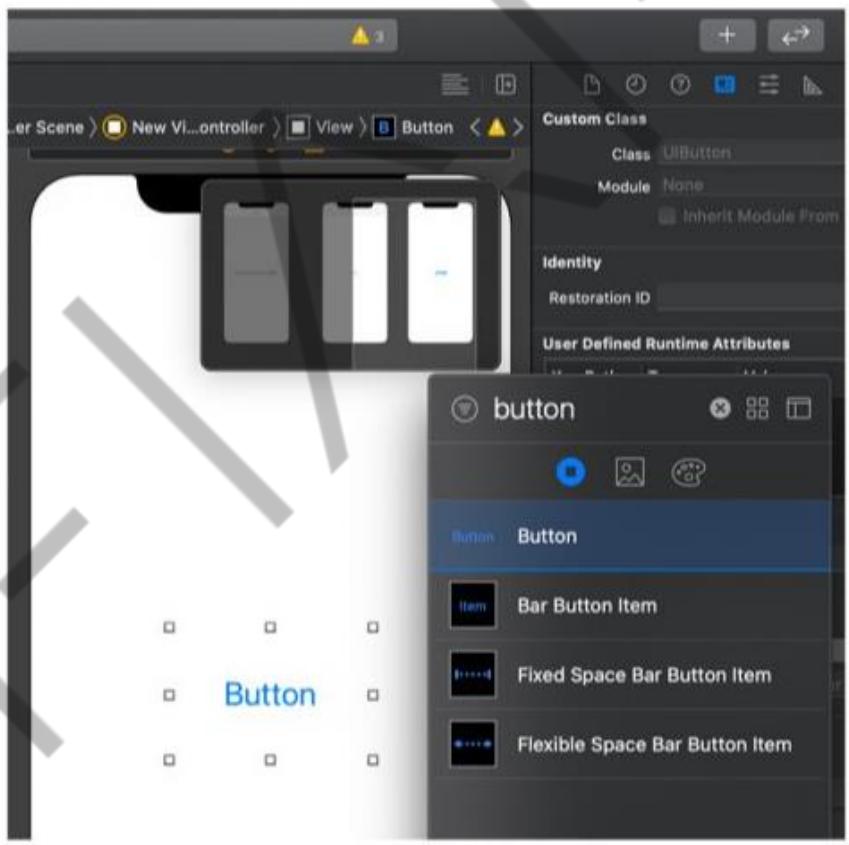

Figura 16 – Exemplos de UIButton
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3 Recuperando informações com UITextField

Quando utilizamos *UILabel*, podemos mostrar uma informação textual para o usuário. Porém, quando precisamos que o usuário nos forneça uma informação em formato de texto, como por exemplo o seu nome ou seu e-mail, como proceder?

Em iOS, o principal componente utilizado para este fim é o **Text Field** (classe *UITextField).* Este componente fornece ao usuário uma área de texto editável, onde é possível inserir textos utilizando o teclado do iOS, que aparece automaticamente sempre que um Text Field recebe o foco. Cabe ao desenvolvedor, via código, fazer com que o teclado desapareça, pois isto não é feito automaticamente. É o componente ideal quando precisamos que nosso usuário entre com pequenas quantidades de texto.

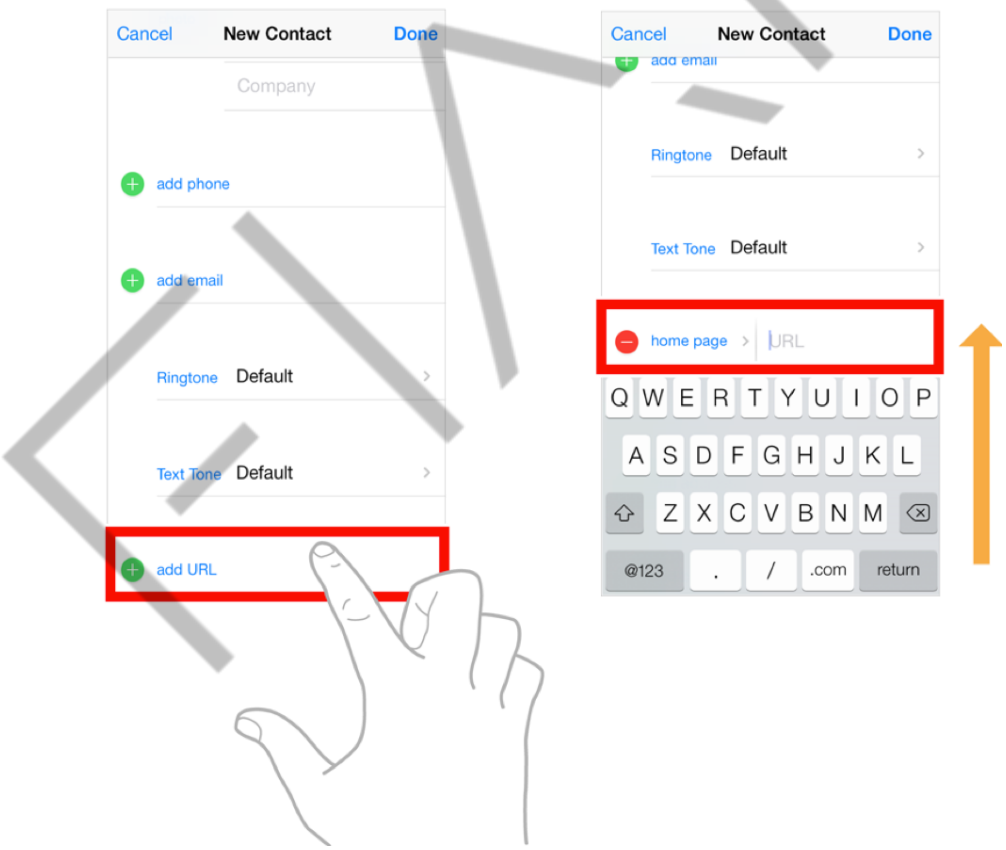

Figura 17 – UItextField sendo utilizado
Fonte: Apple (2018)

Abaixo seguem algumas das suas principais propriedades:

- **Text:** Responsável pelo texto apresentado no campo. É a propriedade que usamos para recuperar o que o usuário digitou.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Color / Font**: Cor e fonte do texto.
- **Alignment**: Alinhamento do texto dentro da área de edição.
- **Placeholder**: Texto apresentado quando o campo estiver vazio. Usado para indicar ao usuário o tipo de informação que o campo utiliza.
- **Background**: Imagem de fundo do textField.
- **Border Style**: Estilo visual do botão (borda).
- **Clear Button**: Define quando o botão clear será apresentado. São quatro opções: *Never appears* (nunca aparece), *Appears while editing* (aparece enquanto editando), *Appears unless editing* (aparece a menos que esteja editando) e *Is always visible* (sempre visível).
- **Keyboard Type**: Tipo do teclado que será apresentado ao usuário, com opções como número, para telefones ou entrada de URL's.
- **Secure text Entry**: Usado quando iremos utilizar senha no campo.

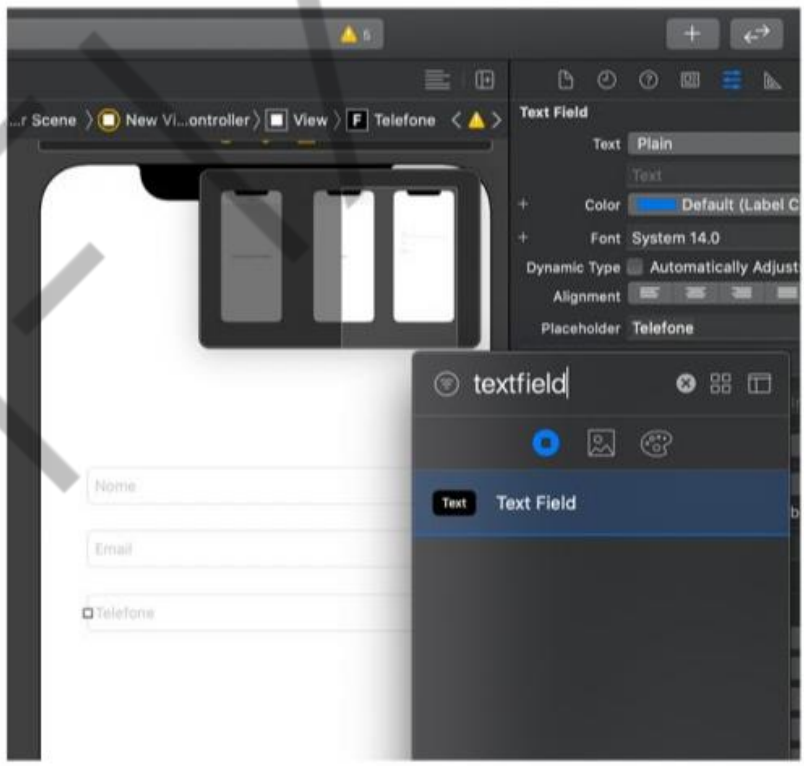

Figura 18 – UItextField
Fonte: Elaborado pelo autor (2021)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.4 Interagindo com o usuário: IBActions e IBOutlets

Aprendemos que a *controller* é a camada que conversa com a *View*, recebendo as interações do usuário e respondendo de acordo. Agora vamos ver na prática como isto ocorre.

Quando a *View* deseja informar à *controller* que o usuário, por exemplo, tocou em um botão, executa uma ação diretamente na *controller.* Esta ação nada mais é do que um método definido pelo desenvolvedor que será chamado de acordo com o evento ocorrido no botão (por exemplo, evento de toque em cima do botão). Isto é feito através do que chamamos de **Actions (IBActions)**.

Quando a *controller* precisa modificar algum componente da *view*, ela precisa ter no código alguma variável que seja uma representação direta daquele componente, e isso é conseguido através de **Outlets (IBOutlets)**.

IBActions e *IBOutlets* são chamados qualificadores, pois qualificam funções e variáveis para interagirem com o *Interface Builder (storyboard).* Daí vem o **IB** do nome (*Interface Builder).*

Para continuar, vamos voltar para o nosso projeto com a Navigation Controller. Apague todo e qualquer elemento que se encontra na sua cena inicial (inclusive o botão que leva para a segunda tela). Adicione um *Label* no topo, com a cor azul (Color), tamanho 32 (Font) e o texto “Bem-Vindo”. Logo abaixo, adicione dois *Text Fields* e defina seus *Placeholder* como Nome e E-mail. Depois, mais abaixo, insira um Button com fundo azul (Background em Custom e Fill em azul), texto branco (Foreground), fonte Semibold e escrito “Entrar”. Abaixo do botão, insira um *Label* com texto vermelho (color), tamanho 17, centralizado (Alignment), escrito “Mensagem de erro” e com número de linhas (Lines) 0. Deixe a cor de fundo da sua tela como cinza. No final, a tela ficará assim:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 19 – Tela com Button, Labels e Text Fields
Fonte: Elaborado pelo autor (2023)

2.4.1 IBOutlet

A ideia é que ao clicar no botão entrar o usuário seja levado para a próxima tela, porém apenas se o mesmo tiver inserido seu nome e e-mail. Para que possamos verificar se os *Text Fields* estão com texto, precisamos ter acesso a eles em nosso código, ou seja, em nossa *controller* (nesse caso, a classe *ViewController*). É através de *IBOutlet* que isso é possível, e para que possamos criar nosso *outlets* precisamos mostrar na tela tanto o *Storyboard* quanto o código. Para isso, selecione a *DragonBall* da nossa primeira cena e clique no *Editor Option* e depois na opção *Assistant* ou use as teclas de atalho COMMAND + OPTION + CONTROL + ENTER. Isso fará com que a tela se divida em duas, mostrando tanto a área visual quanto o código.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

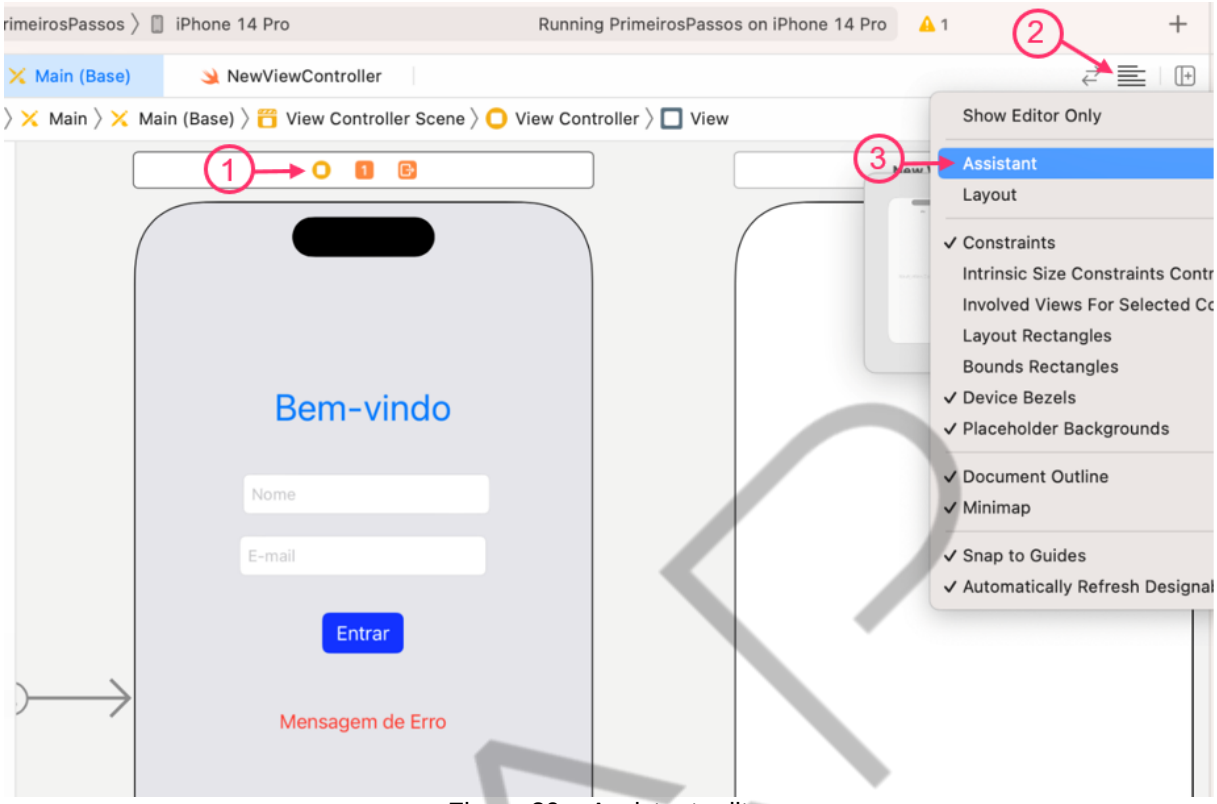

Figura 20 – Assistant editor
Fonte: Elaborado pelo autor (2023)

Para criarmos um *IBOutlets*, precisamos selecionar o componente e com a tecla CTRL pressionada, arrastar até a nossa classe (um procedimento semelhante pode ser feito arrastando o objeto apenas com o botão direito do mouse ou *touch pad*).

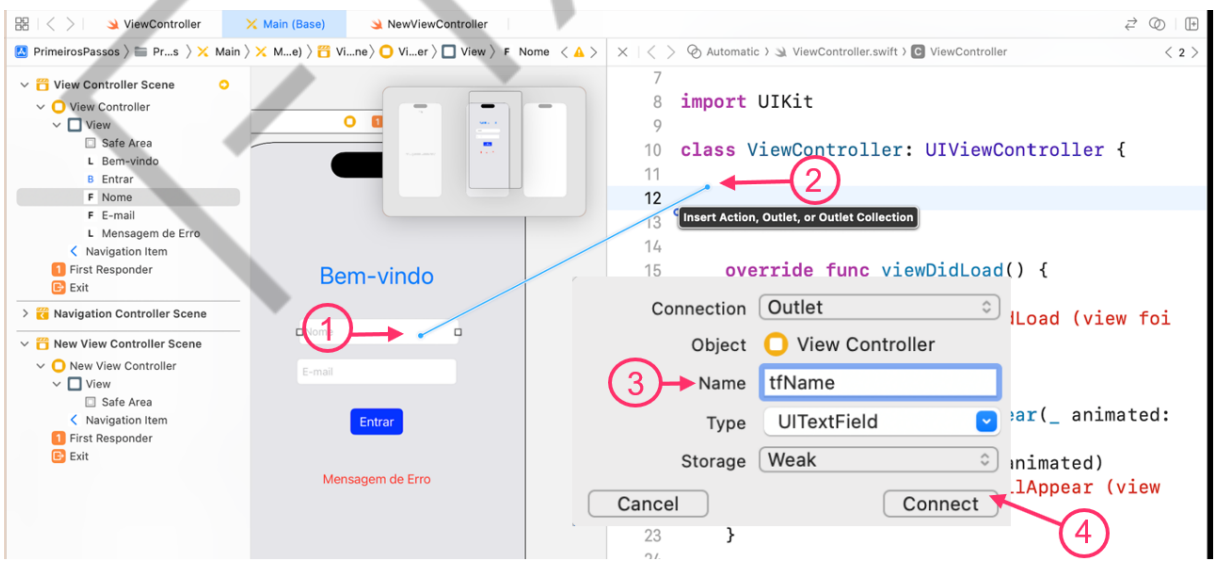

Figura 21 – Criando IBOutlet
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos fazer isso para os dois *Text Fields* e para o *Label* com a mensagem de erro. Só é possível fazer um de cada vez, portanto, selecione o primeiro *Text Field,* segure CTRL e arraste para a classe (posicionando entre *class ViewController* e override func viewDidLoad). Ao soltar, dê o nome do *outlet* de tfName (de textFieldName) e aperte em Connect. Faça o mesmo para o segundo Text Field (tfEmail) e para o *Label* de erro (lbError). No final, o início de sua classe ficará assim:

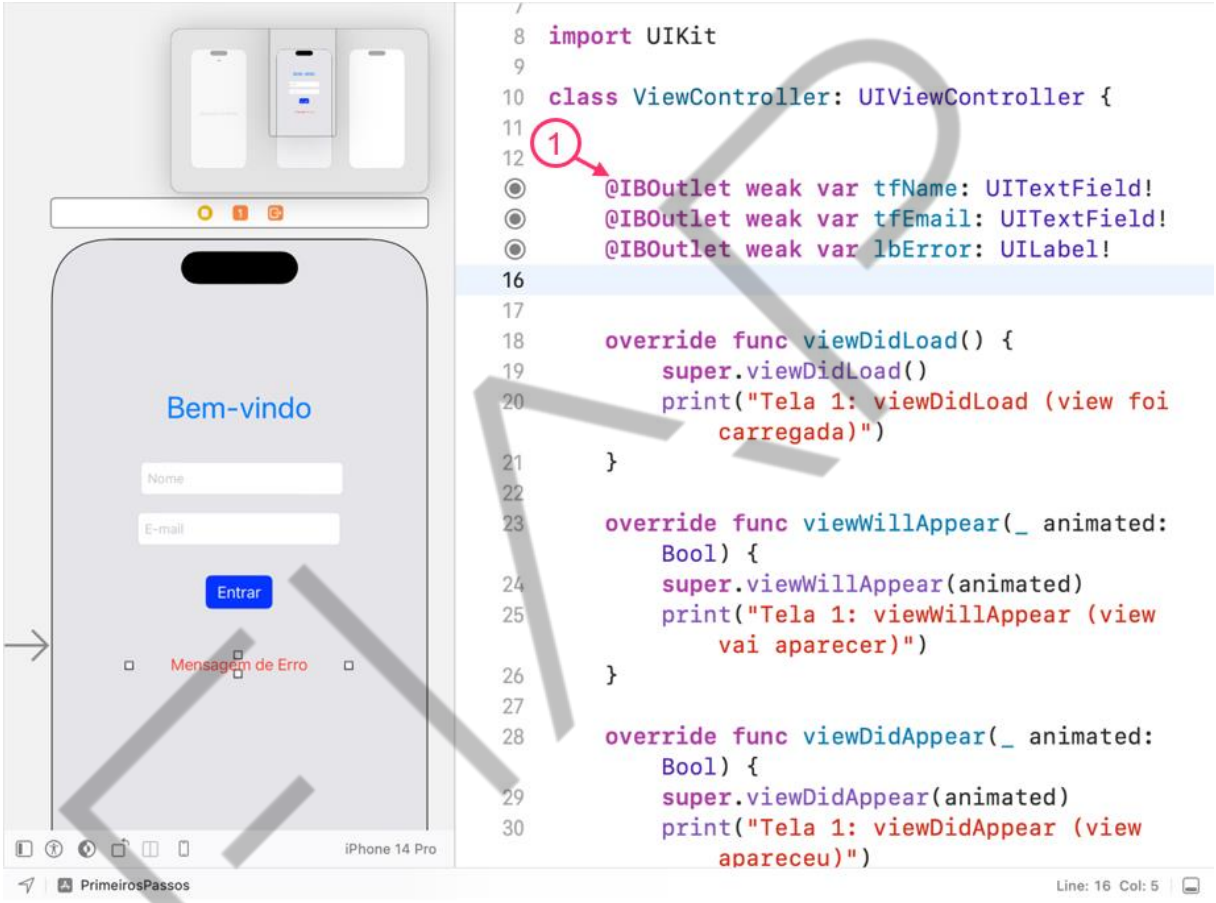

Figura 22 – IBOutlets criados
Fonte: Elaborado pelo autor (2023)

2.4.2 IBAction

Vamos agora criar a *IBAction* do botão, ou seja, a função que será chamada quando o usuário tocar no botão. Clique no botão e segurando a tecla CTRL arraste até o final da classe (antes do último**}**). Na tela que abre, escolha *Action* (caso não esteja) no lugar de *Outlet* e logo abaixo dê o nome da *action* de *checkTextFields*. Em type escolha *UIButton* (para que o método traga uma referência ao botão que o chamou) e clique em **Connect**. Sua classe ficará assim:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

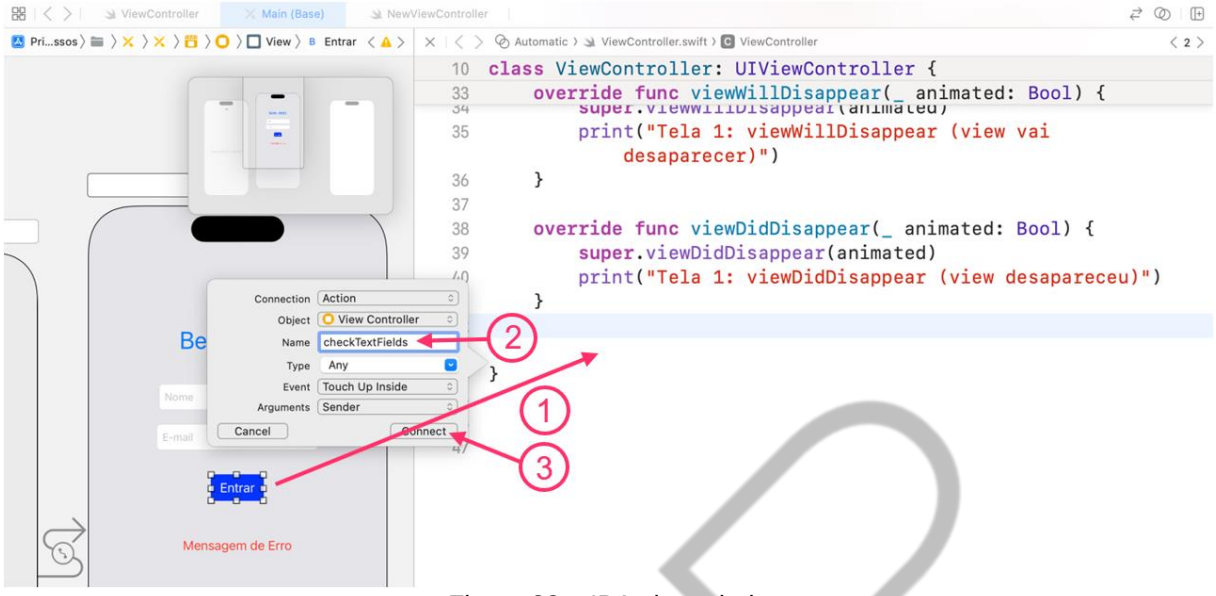

Figura 23 – IBAction criada
Fonte: Elaborado pelo autor (2023)

Agora vamos implementar o código. Ao iniciar a tela, o *label* com o erro deve estar vazio. Para isso, vamos alterar a propriedade text do *label,* pois é ela que contém o texto que estamos lendo.

Depois disso, quando o usuário clicar no botão, vamos verificar se os textos estão vazios, utilizando a propriedade *isEmpty* presente em toda *String,* e que nos devolve um *Bool* dizendo se a *String* está vazia ou não. Se algum dos dois Text Fields estiver vazio, uma mensagem será mostrada na tela (no label de erro). O método *viewDidLoad* e a *action checkTextField* ficarão como no código abaixo.

```
override func viewDidLoad() {
    super.viewDidLoad()
    print("Tela 1: viewDidLoad (view foi carregada)")
    lbError.text = ""
}


@IBAction func checkTextFields(_ sender: UIButton) {
    view.endEditing(true)

    lbError.text = ""
    if tfName.text!.isEmpty || tfEmail.text!.isEmpty {
        lbError.text = "Ambos os campos precisam ser
preenchidos"
    }
}
```

Código-fonte 3 – Manipulando outlets e actions
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Rode seu App e veja que quando a tela aparece, o texto já não está presente, graças ao código inserido na *viewDidLoad.* Além disso, ao tocar no botão Entrar sem ter entrado com todos os dados, o erro aparece. Perceba, também, que o teclado some quando tocamos no botão. Isto ocorre graças ao código **view.endEditing(true)**, que faz com que qualquer edição de textfield seja finalizada e, com isso, o teclado desapareça.

Obs.: Caso seu teclado não apareça automaticamente, aperte COMMAND+K. Este atalho faz com que o teclado, no simulador, apareça ou desapareça. Se estiver rodando o app diretamente em um *device*, o teclado sempre aparecerá quando um Text Field ganhar foco.

2.5 Navegação entre telas (Modal, Navigation Controllers e TabBarControllers)

Vamos entender agora como fazemos para navegar entre telas de um aplicativo. Basicamente existem três formas de fornecermos ao usuário a possibilidade de sair de uma tela e entrar em outra. A primeira delas é a *forma modal* e consiste em mostrar uma tela por cima de toda e qualquer tela presente. A segunda é utilizando *Navigation Controllers,* que é como nosso App está configurado atualmente, onde toda a navegação é feita utilizando esta *controller* especial, que nos dá automaticamente uma maneira bem simples de voltar para a tela anterior. A terceira é quando usamos *Tab Bar Controller,* que permite que possamos navegar entre até cinco telas diferentes. Esta, por exemplo, é a maneira principal que você encontra em aplicativos como o WhatsApp ou Facebook.

Antes de prosseguirmos, é importante saber como é feita a navegação entre telas.

2.5.1 Segue

Quando clicamos em um botão, ou na "*DragonBall"* de uma cena e arrastamos para outra cena (com a tecla CTRL pressionada), o vínculo que criamos entre as duas cenas se chama **Segue**. Uma segue define a conexão entre duas cenas. Define que

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

está saindo de uma cena e entrando em outra. É possível definir um identificador para uma segue de modo que possamos executá-la via código.

2.5.2 Modal

Quando a cena de origem não está sendo controlada através de uma *Navigation Controller*, a apresentação da nova se dá modalmente, ou seja, a nova tela aparece por cima da anterior. Mesmo que você esteja usando *Navigation Controllers* é possível definir uma apresentação modal e para isso você deve escolher a opção **Present Modally** no momento que criar a segue. Para testarmos isso, crie uma segue do botão Entrar até a segunda cena (segure CTRL e arraste até a segunda cena).

No menu que aparece, escolha *Present Modally.* Agora, na classe da segunda cena (NewViewController), implemente o seguinte código depois do método viewDidLoad:

```
override func touchesBegan(_ touches: Set<UITouch>, with
event: UIEvent?) {

    dismiss(animated: true, completion: nil)
}
```

Código-fonte 4 – Fechando tela apresentada modalmente
Fonte: Elaborado pelo autor (2018)

O método *touchesBegan* é um método que é chamado sempre que tocamos na tela (desde que não seja em um botão, Text Field ou qualquer elemento iterativo). O método **dismiss** faz com que uma tela apresentada modalmente seja dispensada. A propriedade *animated* define se será animado ou não, e *completion* seria a *closure* que poderíamos executar ao término da animação.

Rode o App e você verá que a segunda tela aparece por cima da primeira e ao clicar em qualquer lugar dela, você retorna para a anterior.

2.5.3 Navigation Controller

A *Navigation Controller* é um tipo especial de *controller* que serve para coordenar e gerenciar a navegação entre telas. Ela nos fornece uma *Navigation Bar*,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

representada por uma barra no topo da tela que é usada tanto para definir um título quanto abrigar botões de navegação (*back*, ou voltar) que ficam ao lado esquerdo, ou outros botões com ações diversas, que podem se alocar ao lado direito.

Para inserir uma *Navigation Controller (*já fizemos isso em nosso projeto*)* em uma cena basta selecionar a cena (clicando na *DragonBall*) e no menu escolher **Editor -> Embed In -> Navigation Controller**. Este comando faz com que a cena selecionada fique sendo coordenada por uma *Navigation Controller* e a partir desse momento qualquer segue criada (no modo **show**) passa a apresentar a próxima cena através da *Navigation Controller.* Esta é a forma mais usual de se navegar entre telas no iOS.

Como nossa cena está dentro de uma *Navigation Controller,* o que vamos fazer é remover a segue criada anteriormente e criar uma nova. Clique na segue e pressione DELETE no teclado.

Se criarmos a partir do botão, a segunda tela aparecerá quando clicarmos nele (como fizemos anteriormente), porém nesse caso faremos diferente, pois vamos mostrar a segunda cena somente se o usuário preencher o login e a senha. Crie uma segue, porém, não mais a partir do botão entrar e sim a partir da *DragonBall* da primeira cena. Clique nela, arraste (com CTRL) até a segunda cena e escolha **show**.

Com a segue criada, clique na mesma e no *Attributes Inspector* defina a propriedade **Identifier** como **next**.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

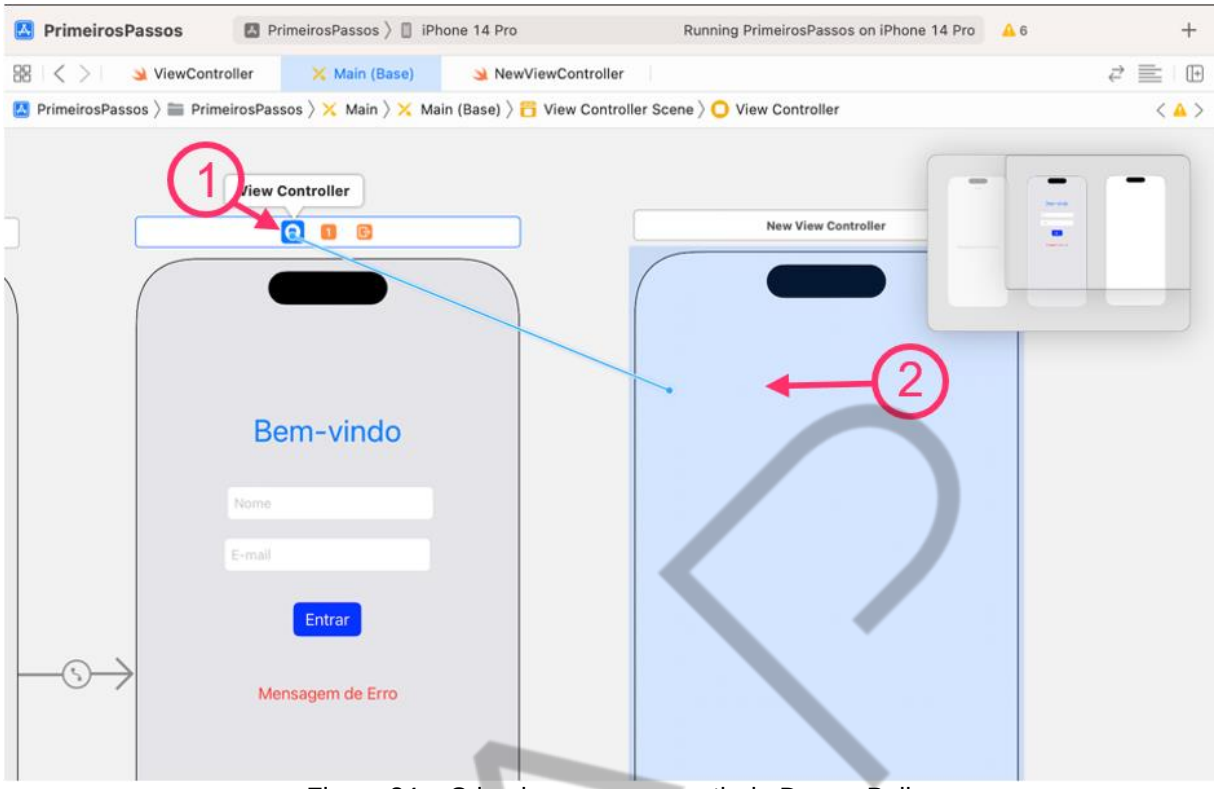

Figura 24 – Criando a segue a partir do DragonBall
Fonte: Elaborado pelo autor (2023)

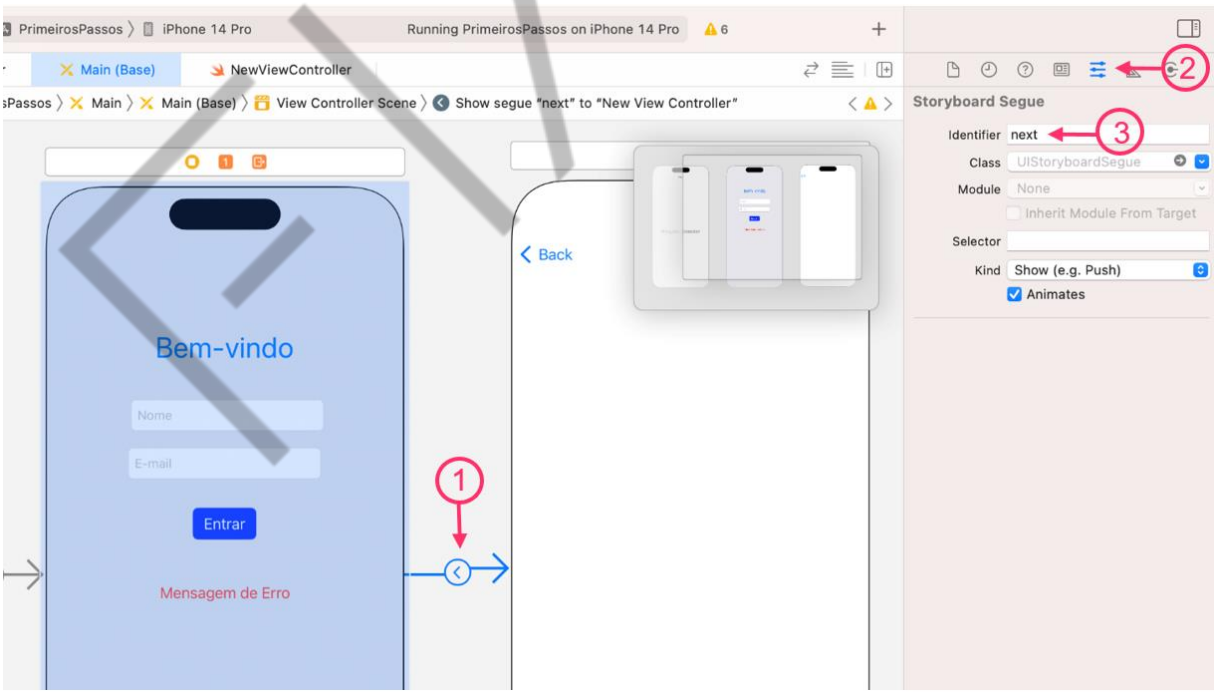

Figura 25 – Definindo identifier da segue
Fonte: Elaborado pelo autor (2023)

Antes de rodar o App, precisamos executar esta segue via código, afinal, não está vinculada ao botão. Usamos o método performSegue(withIdentifier identifier: String, sender: Any?) para isso. Este método executa a segue com o *identifier* que for

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

passado. Modifique o seu método checkTextFields para que fique como no código a seguir.

```
@IBAction func checkTextFields(_ sender: UIButton) {
    view.endEditing(true)
    lbError.text = ""

    if tfName.text!.isEmpty || tfEmail.text!.isEmpty {
        lbError.text = "Ambos os campos precisam ser
preenchidos"
    } else {
        performSegue(withIdentifier: "next", sender: nil)
    }
}
```

Código-fonte 5 – Executando segues via código
Fonte: Elaborado pelo autor (2018)

Agora, se você preencher todos os campos o seu App abrirá a segunda tela.

2.5.4 Tab Bar Controller

Outra *controller* que serve para gerenciar a navegação entre telas é a Tab Bar *Controller*. Com ela, podemos definir até cinco cenas que serão acessadas através de uma barra que fica na parte inferior do aplicativo, chamada de Tab Bar.

Também é possível embutir uma cena dentro de uma *Tab Bar Controller,* da mesma maneira que fizemos com a *Navigation Controller*, porém, no nosso exemplo, vamos adicionar uma *Tab Bar Controller* a partir do seu componente na *Library.* Filtre por *Tab Bar Controller* e arraste o componente que surge para o *Storyboard* (recomendo reduzir o nível de zoom para ter mais espaço antes de fazê-lo).

Você verá que foi adicionado uma *Tab Bar Controller* no *Storyboard* e juntamente com ela, foram adicionadas duas novas cenas, embedadas (embutidas) na *Tab Bar Controller.*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

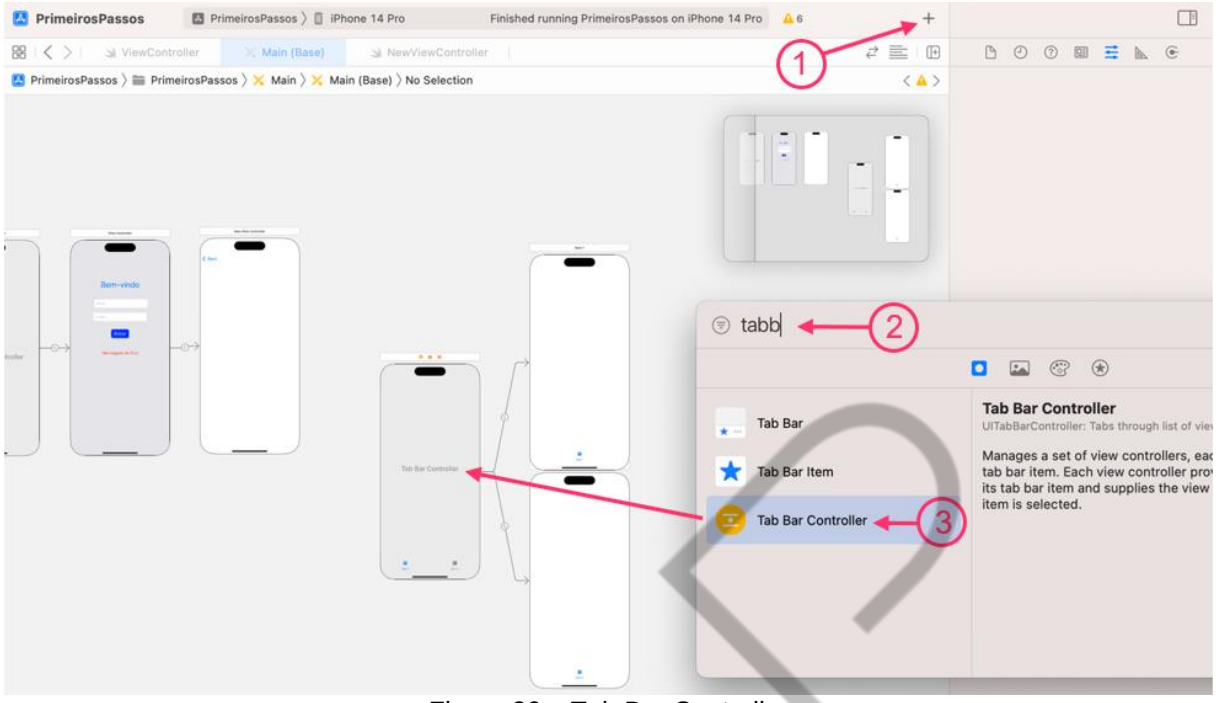

Figura 26 – Tab Bar Controller
Fonte: Elaborado pelo autor (2023)

Podemos alterar o ícone e o texto que ficam embaixo de cada cena. Este componente se chama Tab Bar Item, e ao clicá-lo, podemos definir estas propriedades. Defina o da primeira cena como Tela 1 e o da segunda como Tela 2 (alterando a propriedade **Title**). Aproveite e deixe a cor de fundo da tela 2 como azul.

Se rodar o App agora, nada terá mudado, pois essa *Tab Bar* em questão não tem nenhum vínculo com as telas anteriores. Não vamos fazer este vínculo, o que vamos fazer é definir que nosso *storyboard* vai iniciar nesta *Tab Bar Controller,* e não mais na *Navigation Controller* que temos. Isto pode ser feito definindo qual *controller* será a **Initial View Controller** do *Storyboard.* Selecione a *DragonBall* da *Tab Bar Controller* e no *Attributes Inspector* marque a opção **Is Initial View Controller**. Com isso, você notará que a setinha que estava ao lado da *Navigation Controller* agora está ao lado da *Tab Bar Controller*. Esta seta define qual cena será a cena inicial do *Storyboard,* e você pode também mudá-la de posição simplesmente arrastando de uma cena para outra. Ao final, adicione um objeto button na cena do item 1 e um objeto switch na cena do item 2, isso apenas para diferenciar as duas telas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

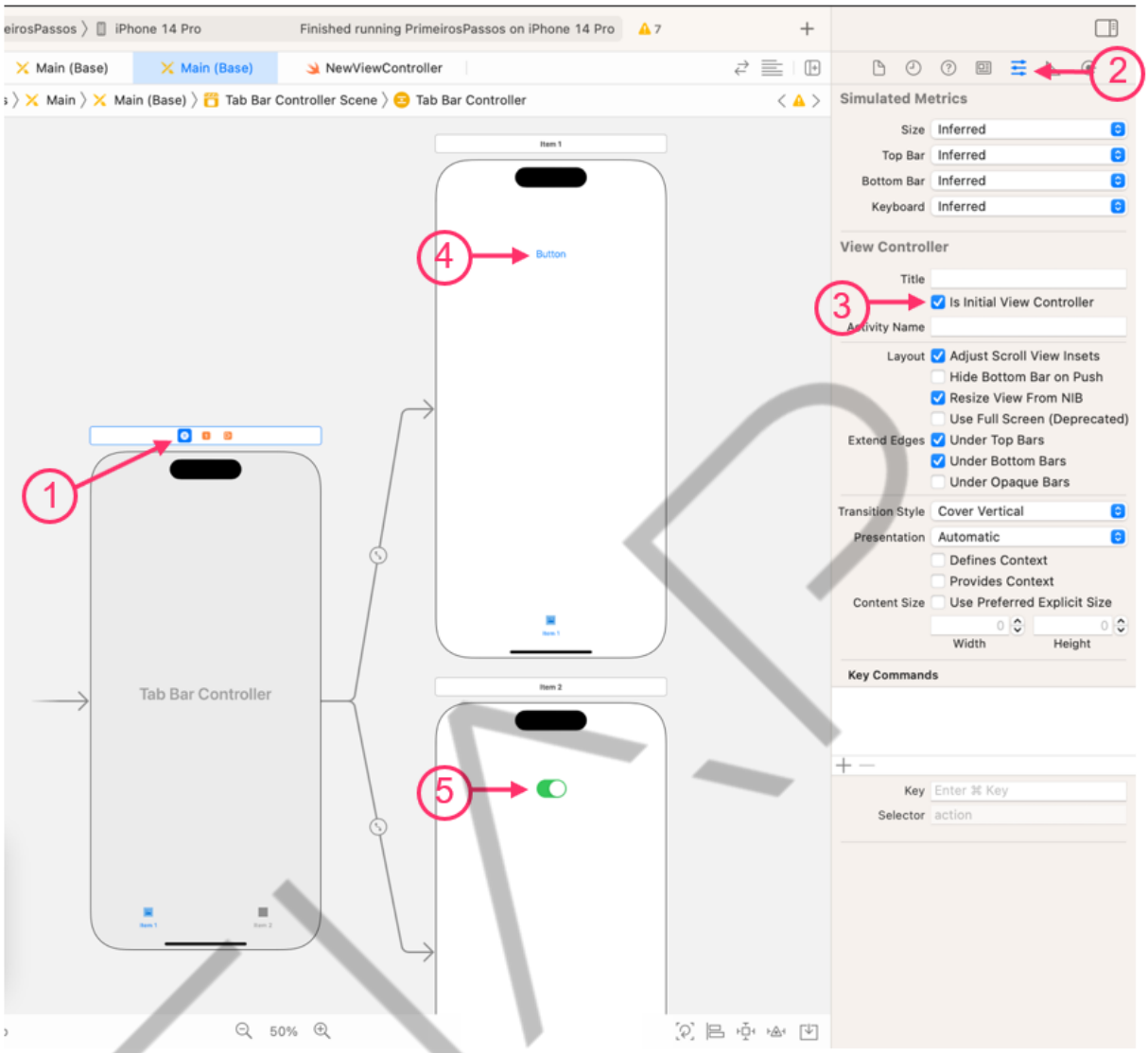

Figura 27 – Definindo como Initial View Controller
Fonte: Elaborado pelo autor (2023)

Rode o *App* e veja como é possível navegar entre duas cenas usando a *Tab Bar Controller*. Se quiser adicionar mais cenas, basta adicionar novas *View Controllers* na tela e para colocá-la dentro da *Tab Bar Controller* você deve clicar com o botão direito no meio da *Tab Bar Controller* e arrastar a bolinha que fica ao lado do item **view controllers**. Este item contém todas as cenas que estão gerenciadas pela *Tab Bar Controller*. No momento aparecem duas cenas, e se você arrastar a bolinha que fica ao lado direito até a nova cena, também fará parte da *Tab Bar Controller.*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

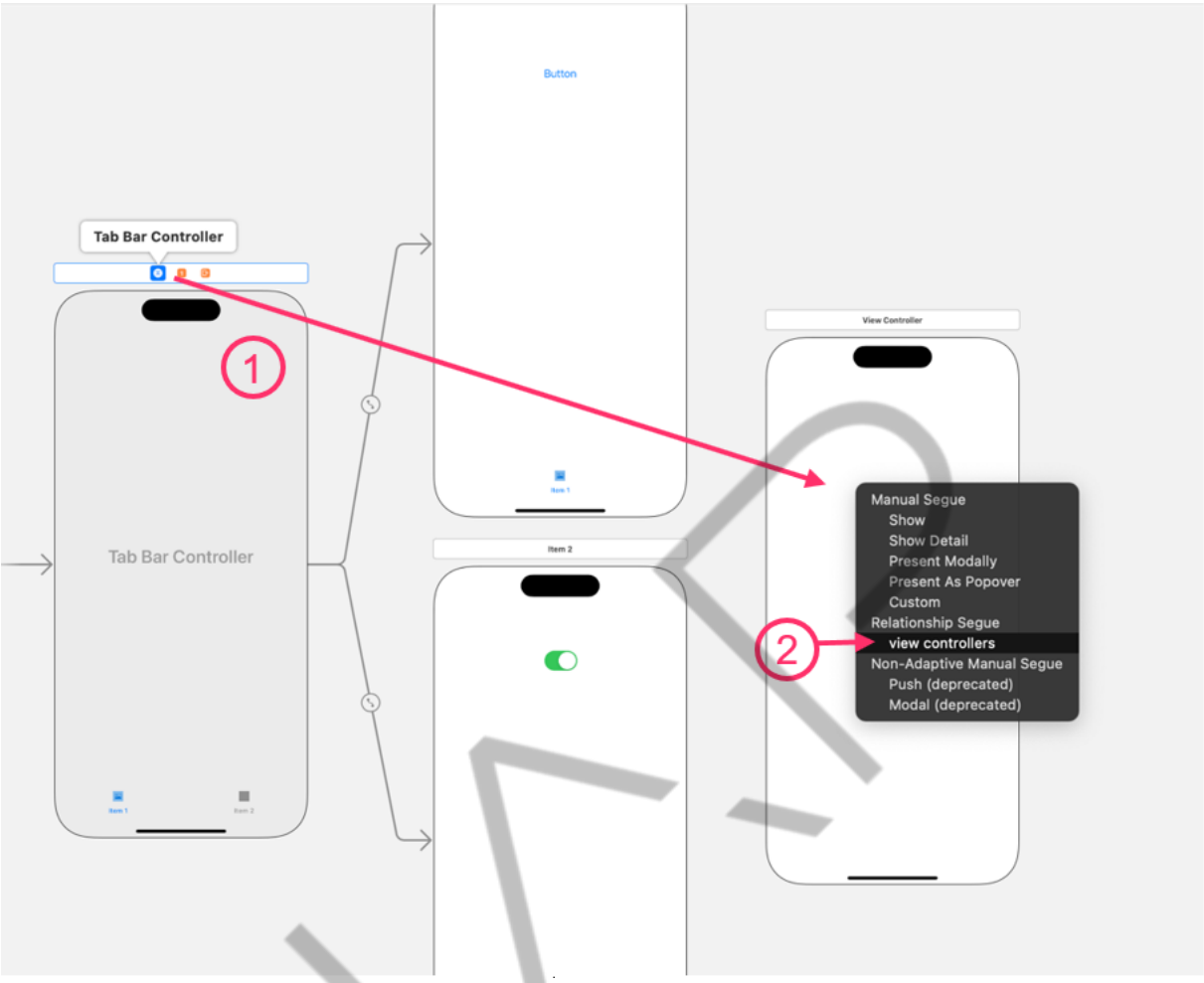

Figura 28 – Adicionando nova cena à Tab Bar Controller
Fonte: Elaborado pelo autor (2023)

Parabéns, agora você já conhece os primeiros passos para a construção de aplicativos iOS. Deste ponto para a frente, avançaremos em conteúdos mais relevantes para o desenvolvimento iOS.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

APPLE. **Framework UIKit** 2023. Disponível em: <https://developer.apple.com/documentation/uikit>. Acesso em: 24 jun. 2023.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com